```
import {
    _decorator, Canvas, Color, Component, director, Director, EventKeyboard, EventTouch, game, Game, Input, inpu
    t,
    KeyCode, Label, Node, ResolutionPolicy, Sprite, sys, UITransform, Vec3, view,
} from 'cc';
import { EDITOR } from 'cc/env';
import { AD_CONFIG } from './AdConfig';
import { Ads, mgApi, RewardResult } from './Ads';
import { Art } from './Art';
import { Backdrop } from './Backdrop';
import { Battle, blessingOptions, describe, Option } from './Battle';
import { ACHIEVEMENTS, BLESSINGS, BlessingDef, BOSS_SCHEDULE, DAILY_GOAL, DAILY_REWARD, dailySetup, activeEvent,
     EVENTS, STAGES, CHARACTERS, CharDef, codexEntries, dailyQuests, QuestDef, QUEST_BONUS, SIGNIN_REWARDS, Code
    xEntry, CodexTab, ENEMIES, POWERUPS, powerUpCost, WEAPONS, WIN_TIME } from './Defs';
import { Recorder } from './Recorder';
import { Save } from './Save';
import { Sound, vibrate } from './Sound';
import { blockTouches, Button, mkButton, mkLabel, mkNode, mkPanel, mkRect, mkSprite } from './UiKit';
const { ccclass } = _decorator;
// 竖屏设计分辨率（适配微信小游戏）
export const DESIGN_W = 720;
export const DESIGN_H = 1280;
// 游戏名与适龄提示（上架前与微信后台填写的保持一致）
export const GAME_NAME = '奶蛙割草';
export const GAME_SUBTITLE = 'MILKY  FROG  SURVIVORS';
export const AGE_RATING = '12+';
export const GAME_VERSION = 'v1.0.0';
// 点击主界面奶蛙时随机说的话
const FROG_LINES = ['呱!', '割草去!', '呱呱呱~', '奶蛙出击!', '今天也要努力割草', '红月? 不怕!', '再来一局呱', '
    别戳了, 痒!'];
const AGE_TIP = [
    '1. 本游戏是一款休闲动作类游戏，适用于年满 12 周岁及以上的用户，',
    '    建议未成年人在家长监护下使用游戏产品。',
    '2. 本游戏为卡通像素画风，角色与怪物均为可爱的卡通形象，',
    '    对抗内容轻度、无血腥恐怖画面，无基于真实历史和现实事件的改编内容。',
    '3. 本游戏无需注册账号，没有充值和付费内容，',
    '    游戏内金币只能通过游玩免费获得，不能交易。',
    '4. 游戏有助于锻炼反应能力与策略规划能力，',
    '    请合理安排游戏时间，注意劳逸结合。',
].join('\n');
const HEALTH_TIP = '抵制不良游戏，拒绝盗版游戏。\n注意自我保护，谨防受骗上当。\n适度游戏益脑，沉迷游戏伤身。\n合
    理安排时间，享受健康生活。';
const C_AD = new Color(150, 90, 20, 255);
type State = 'splash' | 'privacy' | 'menu' | 'daily' | 'chars' | 'shop' | 'settings' | 'achv' | 'codex' | 'signi
    n' | 'play' | 'levelup' | 'chest' | 'pause' | 'revive' | 'result';
const C_WHITE = new Color(255, 255, 255, 255);
const C_GOLD = new Color(255, 214, 90, 255);
const C_RED = new Color(230, 60, 70, 255);
const C_GREY = new Color(170, 165, 190, 255);
const C_GREEN = new Color(110, 230, 120, 255);
```

```
const C_DIM = new Color(0, 0, 0, 170);
const C_BORDER = new Color(214, 170, 70, 255);
const C_BTN_GREEN = new Color(40, 110, 60, 255);
const C_BTN_BLUE = new Color(40, 70, 130, 255);
const C_BTN_GREY = new Color(70, 64, 90, 255);
const tmpV3 = new Vec3();
@ccclass('GameRoot')
export class GameRoot extends Component {
    private state: State = 'menu';
    private vw = 0;
    private vh = 0;
    private safeTop = 0;
    private safeBottom = 0;
    private battle!: Battle;
    private ui!: Node;
    private t = 0;
    // 输入
    private keys = new Set<number>();
    private joyActive = false;
    private joyX = 0;
    private joyY = 0;
    private joyOrigin = new Vec3();
    private touchLayer!: Node;
    private joyBase!: Node;
    private joyKnob!: Node;
    // 界面
    private screens: Partial<Record<State, Node>> = {};
    private overlay!: Sprite;
    private hud!: Node;
    private xpFill!: Node;
    private lvLabel!: Label;
    private timeLabel!: Label;
    private killLabel!: Label;
    private goldLabel!: Label;
    private iconRow!: Node;
    private bossBar!: Node;
    private bossFill!: Node;
    private bossName!: Label;
    private bossTimer!: Label;
    private bannerLabel!: Label;
    private comboLabel!: Label;
    private comboSub!: Label;
    private bannerT = 0;
    private menuHero!: Sprite;
    private menuBackdrop!: Backdrop;
    private splashBackdrop!: Backdrop;
    private menuTitle!: Node;
    private menuTitleY = 0;
    private titleSparks: { node: Node; sp: Sprite; ph: number }[] = [];
    private menuHeroShadow!: Node;
```

```
    private menuHeroScale = 3;
    private menuHeroBaseY = 0;
    private heroJumpY = 0;
    private heroJumpV = 0;
    private bubble!: Node;
    private bubbleLabel!: Label;
    private bubbleT = 0;
    private splashLogo!: Node;
    private splashHero!: Sprite;
    private splashHeroBaseY = 0;
    private splashFill!: Node;
    private splashLoad!: Label;
    private refreshers: (() => void)[] = []; // 各界面的刷新函数
    private cards: { node: Node; icon: Sprite; name: Label; tag: Label; desc: Label; border: Sprite; fill: Sprit
    e }[] = [];
    private cardMode: 'level' | 'blessing' = 'level';
    private blessOpts: BlessingDef[] = [];
    private buffLabel!: Label;
    private options: Option[] = [];
    private levelTitle!: Label;
    private rerollBtn!: Button;
    private chestIcon!: Sprite;
    private chestTitle!: Label;
    private chestDesc!: Label;
    private chestGold!: Label;
    private resultTitle!: Label;
    private resultStats!: Label;
    private resultList!: Node;
    private resultHint!: Label;
    private dailyRun = false;
    private newStage = '';
    private newChar = '';
    private dailyReward = 0;
    // 广告相关
    private adRevived = false;
    private adRerolls = 0;
    private adBlessUsed = false;
    private adGoldUsed = false;
    private adChestUsed = false;
    private adWatchedThisResult = false;
    private lastRunGold = 0;
    private reviveT = 0;
    private reviveCount!: Label;
    private adRerollBtn!: Button;
    private adBlessBtn!: Button;
    private chestAdBtn!: Button;
    private goldAdBtn!: Button;
    private shareBtn!: Button;
    private recBtn!: Button;
    private splashT = 0;
```

```
    private ageNode!: Node;
    private toast!: Label;
    private toastT = 0;
    // 新手引导（首局）：-1 = 不显示
    private tutStep = -1;
    private tutT = 0;
    private tutX0 = 0;
    private tutY0 = 0;
    private tutNode!: Node;
    private tutLabel!: Label;
    private tutBg!: Sprite;
    private tutJoy!: Node;
    private tutKnob!: Node;
    onLoad() {
        view.setDesignResolutionSize(DESIGN_W, DESIGN_H, ResolutionPolicy.FIXED_WIDTH);
        Save.load();
        this.autoLite();
        Art.init();
        Sound.init(this.node);
        Ads.init();
        Recorder.init();
        Recorder.onReady = () => { if (this.state === 'result') this.refreshAll(); };
        const cam = this.getComponent(Canvas)?.cameraComponent;
        if (cam) cam.clearColor = new Color(18, 22, 18, 255);
        this.battle = new Battle(this.node);
        this.battle.onBanner = (t, c) => this.banner(t, c);
        this.ui = mkNode('UI', this.node);
        this.ui.addComponent(UITransform).setContentSize(10, 10);
        this.layout(true);
        input.on(Input.EventType.KEY_DOWN, this.onKeyDown, this);
        input.on(Input.EventType.KEY_UP, this.onKeyUp, this);
        game.on(Game.EVENT_HIDE, this.onHide, this);
        this.setupShare();
        this.battle.reset(this.selectedChar());
        this.battle.setPlayerVisible(false);
        this.splashT = 0;
        this.setState('splash');
    }
    // 首次启动按机型性能自动开启性能模式（安卓微信提供 benchmarkLevel：-1 未知，0~50 越高越好）
    private autoLite() {
        const d = Save.data;
        if (d.liteChecked) return;
        d.liteChecked = true;
        try {
            const w = mgApi();
            const info = w && w.getSystemInfoSync ? w.getSystemInfoSync() : null;
            const lvl = info && typeof info.benchmarkLevel === 'number' ? info.benchmarkLevel : -1;
            if (lvl >= 0 && lvl < 15) d.lite = true;
        } catch (e) { /* 忽略 */ }
        Save.save();
```

```
    }
    onDestroy() {
        input.off(Input.EventType.KEY_DOWN, this.onKeyDown, this);
        input.off(Input.EventType.KEY_UP, this.onKeyUp, this);
        game.off(Game.EVENT_HIDE, this.onHide, this);
    }
    // 分享本局战绩（截取结算画面上半部分作为分享图；按平台规则，分享不发放任何奖励）
    private shareResult() {
        const w = mgApi();
        if (!w || !w.shareAppMessage) { this.showToast('请在微信中打开后分享'); return; }
        const b = this.battle;
        let imageUrl: string | undefined;
        try {
            const cv = (globalThis as any).canvas;
            if (cv && cv.toTempFilePathSync) imageUrl = cv.toTempFilePathSync({ x: 0, y: 0, width: cv.width, hei
    ght: Math.round(cv.width * 0.8), destWidth: 500, destHeight: 400 });
        } catch (e) { /* 截图失败则使用默认分享图 */ }
        const title = b.won ? `我的奶蛙在《${GAME_NAME}》里击败了暗夜魔术师！你也来试试？` : `我的奶蛙坚持了 ${t
    his.fmtTime(b.time)}，割倒 ${b.kills} 只怪物，你能撑多久？`;
        try { w.shareAppMessage({ title, imageUrl }); } catch (e) { /* 忽略 */ }
    }
    private setupShare() {
        const w = mgApi();
        if (!w) return;
        try {
            if (w.showShareMenu) w.showShareMenu({ withShareTicket: true, menus: ['shareAppMessage', 'shareTimel
    ine'] });
            const msg = () => ({ title: Save.data.bestTime ? `${GAME_NAME}：我的奶蛙坚持了 ${this.fmtTime(Save.d
    ata.bestTime)}，你能撑多久？` : `${GAME_NAME}：胖奶蛙割草，在红月下坚持 15 分钟！` });
            if (w.onShareAppMessage) w.onShareAppMessage(msg);
            if (w.onShareTimeline) w.onShareTimeline(msg);
        } catch (e) { /* 忽略 */ }
    }
    // 切到后台自动暂停
    private onHide() {
        if (this.state === 'play') this.setState('pause');
    }
    private selectedChar(): CharDef {
        return CHARACTERS.find(c => c.id === Save.data.selected) || CHARACTERS[0];
    }
    // ---------------------------------------------------------------- 布局
    private computeSafe() {
        let top = 0, bottom = 0;
        try {
            const r = sys.getSafeAreaRect();
            if (r && r.height > 0) {
                top = Math.max(0, this.vh - (r.y + r.height));
                bottom = Math.max(0, r.y);
            }
        } catch (e) { /* 忽略 */ }
```

```
        // 微信右上角胶囊按钮：HUD 放在胶囊下方
        const w = (globalThis as any).wx;
        try {
            if (w && w.getMenuButtonBoundingClientRect) {
                const m = w.getMenuButtonBoundingClientRect();
                const info = w.getSystemInfoSync();
                top = Math.max(top, m.bottom * (this.vw / info.windowWidth) + 8);
            }
        } catch (e) { /* 忽略 */ }
        this.safeTop = Math.min(top, this.vh * 0.15);
        this.safeBottom = Math.min(bottom, this.vh * 0.08);
    }
    private layout(force = false) {
        const vs = view.getVisibleSize();
        if (!force && vs.width === this.vw && vs.height === this.vh) return;
        this.vw = vs.width;
        this.vh = vs.height;
        this.computeSafe();
        this.battle.setViewport(this.vw, this.vh);
        this.buildUI();
        if (!force) {
            this.refreshIcons();
            this.setState(this.state);
        }
    }
    private get top() { return this.vh / 2 - this.safeTop; }
    private get bottom() { return -this.vh / 2 + this.safeBottom; }
    // ---------------------------------------------------------------- 构建界面
    private buildUI() {
        this.ui.destroyAllChildren();
        this.screens = {};
        this.refreshers = [];
        this.cards = [];
        // 触摸层（浮动虚拟摇杆，按下位置即摇杆中心）
        this.touchLayer = mkNode('TouchLayer', this.ui);
        this.touchLayer.addComponent(UITransform).setContentSize(this.vw, this.vh);
        this.touchLayer.on(Node.EventType.TOUCH_START, this.onJoyStart, this);
        this.touchLayer.on(Node.EventType.TOUCH_MOVE, this.onJoyMove, this);
        this.touchLayer.on(Node.EventType.TOUCH_END, this.onJoyEnd, this);
        this.touchLayer.on(Node.EventType.TOUCH_CANCEL, this.onJoyEnd, this);
        this.joyBase = mkSprite(this.ui, 'joy_base', 3).node;
        this.joyKnob = mkSprite(this.ui, 'joy_knob', 3).node;
        this.joyBase.active = this.joyKnob.active = false;
        this.buildHud();
        this.overlay = mkRect(this.ui, this.vw, this.vh, C_DIM, 'Overlay');
        blockTouches(this.overlay.node);
        this.buildMenu();
        this.buildChars();
        this.buildShop();
        this.buildSettings();
```

```
        this.buildPause();
        this.buildLevelUp();
        this.buildChest();
        this.buildResult();
        this.buildAchievements();
        this.buildDaily();
        this.buildCodex();
        this.buildSignin();
        this.buildRevive();
        this.buildSplash();
        this.buildPrivacy();
        this.buildTutorial();
        // 适龄提示弹窗（最上层，启动页与主菜单的标识都能打开）
        this.ageNode = mkPanel(this.ui, 680, 420).node;
        this.ageNode.setPosition(0, this.vh * 0.05);
        mkLabel(this.ageNode, '适龄提示', 34, C_GOLD, 'center', 3).node.setPosition(0, 165);
        mkLabel(this.ageNode, AGE_TIP, 19, C_WHITE, 'left', 2).node.setPosition(-315, 0);
        mkLabel(this.ageNode, '点击关闭', 18, C_GREY, 'center', 2).node.setPosition(0, -180);
        this.ageNode.active = false;
        this.ageNode.on(Node.EventType.TOUCH_END, () => { this.ageNode.active = false; });
        this.toast = mkLabel(this.ui, '', 30, C_WHITE, 'center', 4);
        this.toast.node.setPosition(0, -this.vh * 0.3);
        this.toast.node.active = false;
    }
    private buildHud() {
        const hud = this.hud = mkNode('HUD', this.ui);
        const top = this.top, L = -this.vw / 2, R = this.vw / 2;
        mkRect(hud, this.vw - 12, 24, new Color(24, 18, 40, 230), 'xpBg').node.setPosition(0, top - 16);
        this.xpFill = mkRect(hud, this.vw - 18, 18, new Color(70, 150, 255, 255), 'xpFill', 0, 0.5).node;
        this.xpFill.setPosition(L + 9, top - 16);
        this.lvLabel = mkLabel(hud, 'LV 1', 20, C_WHITE, 'right', 3, true);
        this.lvLabel.node.setPosition(R - 14, top - 16);
        const pause = mkPanel(hud, 72, 72, new Color(40, 34, 70, 230));
        pause.node.setPosition(L + 50, top - 76);
        mkSprite(pause.node, 'icon_pause', 2.5);
        pause.node.on(Node.EventType.TOUCH_END, () => {
            Sound.play('click');
            if (this.state === 'play') this.setState('pause');
        });
        this.timeLabel = mkLabel(hud, '00:00', 48, C_WHITE, 'center', 4, true);
        this.timeLabel.node.setPosition(0, top - 72);
        this.killLabel = mkLabel(hud, '击败 0', 24, C_WHITE, 'right', 3, true);
        this.killLabel.node.setPosition(R - 14, top - 58);
        mkSprite(hud, 'coin', 2.2).node.setPosition(R - 120, top - 92);
        this.goldLabel = mkLabel(hud, '0', 24, C_GOLD, 'right', 3, true);
        this.goldLabel.node.setPosition(R - 14, top - 92);
        this.iconRow = mkNode('Icons', hud);
        // Boss 倒计时（计时器下方）
        this.bossTimer = mkLabel(hud, '', 22, C_WHITE, 'center', 3, true);
        this.bossTimer.node.setPosition(0, top - 114);
```

```
        this.iconRow.setPosition(L + 24, top - 152);
        this.bossBar = mkNode('BossBar', hud);
        this.bossBar.setPosition(0, top - 250);
        mkRect(this.bossBar, 424, 22, new Color(20, 10, 20, 230));
        this.bossFill = mkRect(this.bossBar, 418, 16, new Color(170, 40, 200, 255), 'fill', 0, 0.5).node;
        this.bossFill.setPosition(-209, 0);
        this.bossName = mkLabel(this.bossBar, '', 22, C_WHITE, 'center', 3);
        this.bossName.node.setPosition(0, 28);
        this.bossBar.active = false;
        this.comboLabel = mkLabel(hud, '', 40, C_WHITE, 'right', 4, true);
        this.comboLabel.node.setPosition(R - 20, top - 290);
        this.comboSub = mkLabel(hud, '连击', 22, C_WHITE, 'right', 3);
        this.comboSub.node.setPosition(R - 20, top - 325);
        this.buffLabel = mkLabel(hud, '', 22, new Color(255, 200, 220, 255), 'left', 3);
        this.buffLabel.node.setPosition(L + 16, top - 222);
        this.bannerLabel = mkLabel(hud, '', 44, C_RED, 'center', 5);
        this.bannerLabel.node.setPosition(0, this.vh * 0.16);
        this.bannerLabel.node.active = false;
    }
    // ---------------------------------------------------------------- 新手引导
    private static readonly TUT_TEXT = [
        '按住屏幕任意位置 拖动来移动',
        '武器会自动攻击 靠近小怪试试!',
        '捡起蓝色宝石 攒满经验就能升级',
        '三选一: 新武器 或 强化道具',
        '躲开怪物 坚持 15 分钟就胜利!',
        'Boss 要来了! 看到红圈红线立刻躲开',
    ];
    private buildTutorial() {
        const n = this.tutNode = mkNode('Tutorial', this.ui);
        n.setPosition(0, this.bottom + 230);
        this.tutBg = mkRect(n, this.vw - 40, 70, new Color(20, 12, 34, 210), 'tutBg');
        this.tutLabel = mkLabel(n, '', 30, C_GOLD, 'center', 4);
        this.tutJoy = mkSprite(n, 'joy_base', 3).node;
        this.tutKnob = mkSprite(n, 'joy_knob', 3).node;
        this.tutJoy.setPosition(0, 140);
        n.active = false;
    }
    private tutStart() {
        this.tutStep = !Save.data.tutDone && Save.data.runs === 0 && !this.dailyRun ? 0 : -1;
        this.tutT = 0;
        // 首局开局先不弹命运卡牌，让新手先熟悉移动（第一次升级后补发）
        if (this.tutStep === 0) this.battle.pendingBlessings = 0;
        this.tutX0 = this.battle.px;
        this.tutY0 = this.battle.py;
    }
    private tutNext() {
        this.tutStep++;
        this.tutT = 0;
        // 首局的开局命运卡牌推迟到第一次升级之后再给（先学移动，再学选卡）
```

```
        if (this.tutStep === 4) this.battle.pendingBlessings++;
        if (this.tutStep >= GameRoot.TUT_TEXT.length) this.tutFinish();
    }
    private tutFinish() {
        this.tutStep = -1;
        if (!Save.data.tutDone) { Save.data.tutDone = true; Save.save(); }
    }
    private updateTutorial(dt: number) {
        const step = this.tutStep;
        const st = this.state;
        const show = step >= 0 && (st === 'play' || (st === 'levelup' && step === 3));
        this.tutNode.active = show;
        if (step < 0) return;
        const b = this.battle;
        if (st === 'play' || st === 'levelup') this.tutT += dt;
        // 推进条件
        if (step === 0 && Math.hypot(b.px - this.tutX0, b.py - this.tutY0) > 140) this.tutNext();
        else if (step === 1 && b.kills >= 5) this.tutNext();
        else if (step === 2 && st === 'levelup') this.tutNext();
        else if (step === 3 && st === 'play') this.tutNext();
        else if (step === 4) {
            // 先提示目标，然后隐藏，等第一个 Boss 临近再出现最后一步
            const nb = b.nextBoss();
            if (this.tutT > 4) this.tutNode.active = false;
            if (nb && nb.remain <= 12) this.tutNext();
            else if (!nb) this.tutFinish();
        } else if (step === 5 && this.tutT > 6) this.tutFinish();
        if (this.tutStep < 0) { this.tutNode.active = false; return; }
        const txt = GameRoot.TUT_TEXT[this.tutStep];
        if (this.tutLabel.string !== txt) this.tutLabel.string = txt;
        // 升级界面时引导条放到卡片下方
        this.tutNode.setPosition(0, this.bottom + (st === 'levelup' ? 90 : 230));
        const pulse = 0.85 + 0.15 * Math.sin(this.t * 6);
        this.tutLabel.node.setScale(pulse * 0.1 + 0.9, pulse * 0.1 + 0.9, 1);
        this.tutLabel.color = this.tutStep === 5 ? new Color(255, 110, 100, 255) : C_GOLD;
        // 第一步：虚拟摇杆演示动画
        const demo = this.tutStep === 0 && st === 'play';
        this.tutJoy.active = this.tutKnob.active = demo;
        if (demo) {
            const a = this.t * 2.2;
            this.tutKnob.setPosition(Math.cos(a) * 34, 140 + Math.sin(a * 2) * 18);
        }
    }
    // 适龄提示标识（CADPA 规范：12+ 为蓝色，放在左上角，点击查看适龄提示）
    private ageBadge(parent: Node, x: number, y: number) {
        const p = mkPanel(parent, 84, 88, new Color(0, 112, 192, 255), new Color(255, 255, 255, 255));
        p.node.setPosition(x, y);
        mkLabel(p.node, AGE_RATING, 34, C_WHITE, 'center', 0).node.setPosition(0, 10);
        mkLabel(p.node, 'CADPA', 15, C_WHITE, 'center', 0).node.setPosition(0, -26);
        p.node.on(Node.EventType.TOUCH_END, () => { Sound.play('click'); this.showAgeTip(); });
```

```
        return p;
    }
    private showAgeTip() {
        if (this.ageNode) this.ageNode.active = true;
    }
    private screen(state: State): Node {
        const n = mkNode('Screen_' + state, this.ui);
        this.screens[state] = n;
        return n;
    }
    private header(parent: Node, title: string, withGold = true) {
        mkLabel(parent, title, 52, C_GOLD, 'center', 5).node.setPosition(0, this.top - 60);
        if (!withGold) return;
        mkSprite(parent, 'coin', 3).node.setPosition(-50, this.top - 122);
        const g = mkLabel(parent, '', 32, C_GOLD, 'left', 3);
        g.node.setPosition(-28, this.top - 122);
        this.refreshers.push(() => { g.string = String(Save.data.gold); });
    }
    private buildMenu() {
        const s = this.screen('menu');
        const top = this.top;
        const btnTop = this.bottom + 120 + 4 * 100;
        const horizon = btnTop + 70;
        const titleY = this.menuTitleY = top - 180;
        // 动态夜景背景
        this.menuBackdrop = new Backdrop(s, this.vw, this.vh, horizon, titleY - 10);
        // 标题：投影 + 描边 + 闪光
        this.menuTitle = mkNode('title', s);
        this.menuTitle.setPosition(0, titleY);
        mkLabel(this.menuTitle, GAME_NAME, 104, new Color(60, 6, 16, 255), 'center', 0).node.setPosition(6, -8);
        mkLabel(this.menuTitle, GAME_NAME, 104, new Color(255, 236, 120, 255), 'center', 8);
        mkLabel(this.menuTitle, GAME_SUBTITLE, 28, new Color(255, 120, 110, 255), 'center', 4).node.setPosition(
0, -84);
        this.titleSparks = [];
        for (const [x, y] of [[-250, 50], [235, 60], [-205, -55], [262, -30], [-120, 78], [140, -78]]) {
            const sp = mkSprite(this.menuTitle, 'star', 2.4);
            sp.node.setPosition(x, y);
            this.titleSparks.push({ node: sp.node, sp, ph: Math.random() * 6.28 });
        }
        // 主角（只显示当前角色）：根据可用空间自适应大小，点击会跳一下并说话
        const heroH = Art.f('hero_hunter_0').rect.height;
        const avail = titleY - 130 - horizon;
        const sc = this.menuHeroScale = Math.max(1.8, Math.min(3.4, (avail - 30) / heroH));
        this.menuHeroBaseY = horizon - 6 + (heroH * sc) / 2;
        this.menuHeroShadow = mkSprite(s, 'shadow', sc * 1.7).node;
        this.menuHeroShadow.setPosition(0, horizon - 4);
        this.menuHero = mkSprite(s, 'hero_hunter_0', sc);
        this.menuHero.node.setPosition(0, this.menuHeroBaseY);
        this.menuHero.node.on(Node.EventType.TOUCH_END, () => this.heroHop());
        // 说话气泡
```

```
    this.bubble = mkPanel(s, 240, 60, new Color(255, 255, 255, 250), new Color(30, 20, 40, 255)).node;
    this.bubbleLabel = mkLabel(this.bubble, '', 26, new Color(40, 24, 30, 255), 'center', 0);
    mkRect(this.bubble, 14, 14, new Color(255, 255, 255, 250), 'tail').node.setPosition(-60, -34);
    this.bubble.active = false;
    // 角色名牌（点击进入角色选择）
    const plate = mkPanel(s, 230, 52, new Color(36, 20, 50, 235));
    plate.node.setPosition(this.vw / 2 - 135, horizon + 34);
    const name = mkLabel(plate.node, '', 22, C_WHITE, 'center', 2);
    plate.node.on(Node.EventType.TOUCH_END, () => { Sound.play('click'); this.setState('chars'); });
    // 顶部：金币、每日奖励、适龄
    const goldPanel = mkPanel(s, 200, 56, new Color(36, 20, 50, 235));
    goldPanel.node.setPosition(this.vw / 2 - 116, top - 40);
    mkSprite(goldPanel.node, 'coin', 3).node.setPosition(-68, 0);
    const gold = mkLabel(goldPanel.node, '', 28, C_GOLD, 'left', 3);
    gold.node.setPosition(-46, 0);
    const daily = mkLabel(s, '今日首局 金币×2', 20, new Color(255, 214, 90, 255), 'left', 3);
    daily.node.setPosition(-this.vw / 2 + 108, top - 40);
    // 限时活动提示
    const ev = mkLabel(s, '', 22, new Color(255, 120, 90, 255), 'center', 3);
    ev.node.setPosition(0, titleY - 128);
    this.refreshers.push(() => {
        const cur = activeEvent();
        ev.node.active = !!cur;
        if (cur && ev.string !== cur.badge) ev.string = cur.badge;
    });
    this.ageBadge(s, -this.vw / 2 + 56, top - 48);
    const best = mkLabel(s, '', 22, C_GREY, 'center', 2);
    best.node.setPosition(0, this.bottom + 46);
    mkLabel(s, GAME_VERSION, 16, new Color(120, 110, 140, 255), 'left', 2).node.setPosition(-this.vw / 2 + 1
2, this.bottom + 14);
    mkButton(s, '开始游戏', 440, 100, () => this.startGame(), C_BTN_GREEN, 40).node.setPosition(0, btnTop);
    // 签到入口（右上角适龄标识下方），未签到时亮红点
    const sbtn = mkButton(s, '签到', 120, 60, () => this.setState('signin'), new Color(150, 60, 30, 255), 26
);
    sbtn.node.setPosition(this.vw / 2 - 80, this.top - 110);
    const sdot = mkRect(sbtn.node, 16, 16, new Color(240, 60, 60, 255), 'dot');
    sdot.node.setPosition(52, 22);
    this.refreshers.push(() => { sdot.node.active = this.signDue(); });
    const dbtn = mkButton(s, '每日挑战', 212, 84, () => this.setState('daily'), new Color(150, 60, 30, 255),
 32);
    dbtn.node.setPosition(-114, btnTop - 104);
    // 今日挑战未完成时显示红点
    const ddot = mkRect(dbtn.node, 18, 18, new Color(240, 60, 60, 255), 'dot');
    ddot.node.setPosition(94, 30);
    this.refreshers.push(() => { ddot.node.active = Save.data.dailyDone !== this.today; });
    mkButton(s, '选择角色', 212, 84, () => this.setState('chars'), C_BTN_BLUE, 32).node.setPosition(114, btn
Top - 104);
    mkButton(s, '永久强化', 440, 84, () => this.setState('shop'), C_BTN_BLUE, 34).node.setPosition(0, btnTop
 - 200);
    mkButton(s, '成就', 140, 80, () => this.setState('achv'), C_BTN_BLUE, 30).node.setPosition(-150, btnTop
```

```
- 296);
    const cb = mkButton(s, '图鉴', 140, 80, () => this.setState('codex'), C_BTN_BLUE, 30);
    cb.node.setPosition(0, btnTop - 296);
    // 有新收录时在图鉴按钮上显示红点
    const dot = mkRect(cb.node, 18, 18, new Color(240, 60, 60, 255), 'dot');
    dot.node.setPosition(58, 28);
    this.refreshers.push(() => { dot.node.active = Save.data.codex.length > (Save.data.codexSeen || 0); });
    mkButton(s, '设置', 140, 80, () => this.setState('settings'), C_BTN_GREY, 30).node.setPosition(150, btnT
op - 296);
    // 适龄提示面板（最后创建，位于最上层）
    this.refreshers.push(() => {
        const c = this.selectedChar();
        this.menuHero.spriteFrame = Art.f(c.sprite + '_0');
        name.string = `${c.name} · ${c.title} ›`;
        gold.string = String(Save.data.gold);
        daily.node.active = Save.data.bonusDay !== new Date().toDateString();
        const d = Save.data;
        best.string = (d.runs ? `最佳：存活 ${this.fmtTime(d.bestTime)}  ·  击败 ${d.bestKills}  ·  等级 $
{d.bestLevel}` : '存活 15 分钟即可获胜') + (d.endlessBest ? `\n无尽模式最佳 ${this.fmtTime(d.endlessBest)}`
: '') + `   成就 ${d.achievements.length}/${ACHIEVEMENTS.length}`;
    });
}
// 点击主角：跳一下、说句话、呱一声
private heroHop() {
    if (this.heroJumpY > 0) return;
    this.heroJumpV = 760;
    this.bubbleLabel.string = FROG_LINES[Math.floor(Math.random() * FROG_LINES.length)];
    this.bubbleT = 1.6;
    Sound.play('croak');
    vibrate();
}
private updateMenuFx(dt: number) {
    this.menuBackdrop.update(dt);
    const t = this.t;
    this.menuTitle.setPosition(0, this.menuTitleY + Math.sin(t * 1.6) * 6);
    const ts = 1 + Math.sin(t * 2.2) * 0.012;
    this.menuTitle.setScale(ts, ts, 1);
    for (const sp of this.titleSparks) {
        const k = 0.5 + 0.5 * Math.sin(t * 3 + sp.ph);
        sp.sp.color = new Color(255, 255, 255, Math.round(255 * k));
        const sc = 1.6 + 1.2 * k;
        sp.node.setScale(sc, sc, 1);
    }
    // 主角：跳跃物理 + 呼吸动画
    if (this.heroJumpV !== 0 || this.heroJumpY > 0) {
        this.heroJumpV -= 2400 * dt;
        this.heroJumpY += this.heroJumpV * dt;
        if (this.heroJumpY <= 0) { this.heroJumpY = 0; this.heroJumpV = 0; }
    }
    const c = this.selectedChar();
```

```
        const f = this.heroJumpY > 0 ? 2 : Math.floor(t / 0.45) % 2;
        this.menuHero.spriteFrame = Art.f(c.sprite + '_' + f);
        this.menuHero.node.setPosition(0, this.menuHeroBaseY + this.heroJumpY);
        const ss = this.menuHeroScale * 1.7 * Math.max(0.5, 1 - this.heroJumpY / 260);
        this.menuHeroShadow.setScale(ss, this.menuHeroScale * 1.3 * Math.max(0.5, 1 - this.heroJumpY / 260), 1);
        if (this.bubbleT > 0) {
            this.bubbleT -= dt;
            this.bubble.active = this.bubbleT > 0;
            const hh = Art.f(c.sprite + '_0').rect.height * this.menuHeroScale;
            this.bubble.setPosition(110, this.menuHeroBaseY + this.heroJumpY + hh / 2 + 10);
        }
    }
    private buildChars() {
        const s = this.screen('chars');
        this.header(s, '选择角色');
        // 三列紧凑卡片：立绘 / 名字 / 初始武器 / 天赋
        const y0 = this.top - 170;
        const cols = 3;
        const rowsN = Math.ceil(CHARACTERS.length / cols);
        const avail = y0 - (this.bottom + 310);
        const ch = Math.min(250, avail / rowsN - 10);
        const cw = 226;
        CHARACTERS.forEach((c, i) => {
            const col = i % cols, row = Math.floor(i / cols);
            const card = mkPanel(s, cw, ch);
            card.node.setPosition((col - 1) * (cw + 10), y0 - ch / 2 - row * (ch + 10));
            const sc = Math.min(1.5, (ch - 96) / 68);
            const spr = mkSprite(card.node, c.sprite + '_0', sc);
            spr.node.setPosition(0, ch / 2 - 8 - 34 * sc);
            mkLabel(card.node, c.name, 26, C_WHITE, 'center', 3).node.setPosition(0, -ch / 2 + 70);
            const wd = WEAPONS.find(w => w.id === c.weapon)!;
            mkSprite(card.node, wd.icon, 1.6).node.setPosition(-cw / 2 + 30, -ch / 2 + 42);
            mkLabel(card.node, wd.name, 18, C_GOLD, 'left', 2).node.setPosition(-cw / 2 + 46, -ch / 2 + 42);
            mkLabel(card.node, c.desc, 15, C_GREEN, 'center', 2).node.setPosition(0, -ch / 2 + 16);
            const lock = mkRect(card.node, cw - 6, ch - 6, new Color(0, 0, 0, 175), 'lock');
            mkSprite(lock.node, 'icon_lock', 3).node.setPosition(0, 20);
            const evOf = EVENTS.find(e => e.char === c.id);
            mkLabel(lock.node, c.cost < 0 ? (evOf ? evOf.lockText : '活动限定') : `${c.cost} 金币解锁`, 22, C_GO
LD, 'center', 3).node.setPosition(0, -34);
            card.node.on(Node.EventType.TOUCH_END, () => {
                Sound.play('click');
                const d = Save.data;
                if (!d.chars.includes(c.id)) {
                    if (c.cost < 0) { this.showToast(evOf ? evOf.hint : '活动限定角色'); return; }
                    if (d.gold < c.cost) { this.showToast('金币不足'); return; }
                    d.gold -= c.cost;
                    d.chars.push(c.id);
                    Sound.play('chest');
                    this.showToast(`解锁 ${c.name}!`);
                }
```

```
            d.selected = c.id;
            Save.save();
            this.refreshAll();
        });
        this.refreshers.push(() => {
            lock.node.active = !Save.data.chars.includes(c.id);
            const sel = Save.data.selected === c.id;
            card.border.color = sel ? C_GREEN : C_BORDER;
            card.fill.color = sel ? new Color(36, 64, 60, 250) : new Color(36, 30, 64, 245);
        });
    });
    const by = this.bottom + 80;
    mkButton(s, '返回', 260, 88, () => this.setState('menu'), C_BTN_GREY).node.setPosition(-150, by);
    mkButton(s, '出发!', 260, 88, () => this.startGame(), C_BTN_GREEN, 36).node.setPosition(150, by);
    // 模式切换：无尽模式在首次胜利后解锁
    const mode = mkButton(s, '', 560, 64, () => {
        if (Save.data.wins <= 0) { this.showToast('首次胜利后解锁无尽模式与噩梦难度'); return; }
        const order: ('normal' | 'endless' | 'nightmare')[] = ['normal', 'endless', 'nightmare'];
        Save.data.mode = order[(order.indexOf(Save.data.mode) + 1) % order.length];
        Save.save();
        this.refreshAll();
    }, C_BTN_GREY, 26);
    mode.node.setPosition(0, by + 92);
    // 地图选择：点击切换到下一张已解锁的地图
    const stageBtn = mkButton(s, '', 560, 64, () => {
        const d = Save.data;
        const unlocked = STAGES.filter(st => this.stageOpen(st.id));
        const i = unlocked.findIndex(st => st.id === d.stage);
        d.stage = unlocked[(i + 1) % unlocked.length].id;
        if (unlocked.length === 1) this.showToast('在月光草原胜利后解锁新地图');
        Save.save();
        this.refreshAll();
    }, new Color(40, 70, 90, 255), 24);
    stageBtn.node.setPosition(0, by + 166);
    this.refreshers.push(() => {
        const st = this.currentStage;
        // 计数只算常驻地图（活动地图不计入分母）
        const regular = STAGES.filter(x => !x.event);
        const n = regular.filter(x => this.stageOpen(x.id)).length;
        stageBtn.label.string = `地图：${st.name}${st.event ? '（活动限定）' : `（${n}/${regular.length}）`}
 ${st.hpMul > 1 ? `生命+${Math.round((st.hpMul - 1) * 100)}% 金币+${Math.round((st.goldMul - 1) * 100)}%` :
'标准难度'}`;
    });
    this.refreshers.push(() => {
        const locked = Save.data.wins <= 0;
        const e = this.endlessMode, n = this.nightmareMode;
        mode.label.string = locked ? '模式：普通（无尽 / 噩梦 首胜解锁）' : e ? '模式：无尽（Boss 循环，越来
越强）' : n ? '模式：噩梦（生命 +60% 伤害 +30% 金币 +60%）' : '模式：普通（存活 15 分钟胜利）';
        mode.bg.color = e ? new Color(110, 40, 120, 255) : n ? new Color(140, 30, 40, 255) : C_BTN_GREY;
    });
```

```
    }
    private buildShop() {
        const s = this.screen('shop');
        this.header(s, '永久强化');
        const rows = Math.ceil(POWERUPS.length / 2);
        const y0 = this.top - 160;
        const avail = y0 - (this.bottom + 150);
        const ch = Math.min(130, avail / rows - 10);
        const cw = 344;
        POWERUPS.forEach((p, i) => {
            const col = i % 2, row = Math.floor(i / 2);
            const card = mkPanel(s, cw, ch);
            card.node.setPosition((col - 0.5) * (cw + 12), y0 - ch / 2 - row * (ch + 10));
            mkSprite(card.node, p.icon, Math.min(3, ch / 40)).node.setPosition(-cw / 2 + 38, 0);
            mkLabel(card.node, p.name, 26, C_WHITE, 'left', 3).node.setPosition(-cw / 2 + 72, ch * 0.25);
            mkLabel(card.node, p.desc, 17, C_GREY, 'left', 2).node.setPosition(-cw / 2 + 72, -ch * 0.3);
            const pips: Sprite[] = [];
            for (let k = 0; k < p.max; k++) {
                const pip = mkRect(card.node, 14, 14, C_GREY, 'pip');
                pip.node.setPosition(-cw / 2 + 80 + k * 20, 0);
                pips.push(pip);
            }
            const cost = mkLabel(card.node, '', 22, C_GOLD, 'right', 3);
            cost.node.setPosition(cw / 2 - 12, ch * 0.25);
            card.node.on(Node.EventType.TOUCH_END, () => {
                const r = Save.rank(p.id);
                if (r >= p.max) { Sound.play('click'); this.showToast('已满级'); return; }
                const c = powerUpCost(p, r, Save.totalBought());
                if (Save.data.gold < c) { Sound.play('click'); this.showToast('金币不足'); return; }
                Save.data.gold -= c;
                Save.data.power[p.id] = r + 1;
                Save.save();
                Sound.play('coin');
                this.refreshAll();
            });
            this.refreshers.push(() => {
                const r = Save.rank(p.id);
                pips.forEach((pp, k) => { pp.color = k < r ? C_GOLD : new Color(70, 64, 90, 255); });
                cost.string = r >= p.max ? '已满' : String(powerUpCost(p, r, Save.totalBought()));
            });
        });
        const by = this.bottom + 80;
        mkButton(s, '全部返还', 260, 88, () => this.refundAll(), C_BTN_GREY, 30).node.setPosition(-150, by);
        mkButton(s, '返回', 260, 88, () => this.setState('menu'), C_BTN_BLUE).node.setPosition(150, by);
    }
    // 按购买价格全额返还所有强化
    private refundAll() {
        const d = Save.data;
        const ranks: Record<string, number> = { ...d.power };
        let bought = Save.totalBought();
```

```
        let total = 0;
        while (bought > 0) {
            const p = POWERUPS.find(x => (ranks[x.id] || 0) > 0);
            if (!p) break;
            ranks[p.id]--;
            bought--;
            total += powerUpCost(p, ranks[p.id], bought);
        }
        if (!total) { this.showToast('没有可返还的强化'); return; }
        d.gold += total;
        d.power = {};
        Save.save();
        Sound.play('coin');
        this.showToast(`返还 ${total} 金币`);
        this.refreshAll();
    }
    private toggleRow(parent: Node, y: number, icon: string, text: string, key: 'music' | 'sfx' | 'vibrate' | 'l
    ite') {
        mkSprite(parent, icon, 3).node.setPosition(-210, y);
        mkLabel(parent, text, 32, C_WHITE, 'left', 3).node.setPosition(-165, y);
        const btn = mkButton(parent, '', 150, 72, () => {
            Save.data[key] = !Save.data[key];
            Save.save();
            if (key === 'music') Sound.refreshMusic();
            this.refreshAll();
        }, C_BTN_GREEN, 30);
        btn.node.setPosition(170, y);
        this.refreshers.push(() => {
            const on = Save.data[key];
            btn.label.string = on ? '开' : '关';
            btn.bg.color = on ? C_BTN_GREEN : C_BTN_GREY;
        });
    }
    private buildSettings() {
        const s = this.screen('settings');
        this.header(s, '设置', false);
        const p = mkPanel(s, 580, 470);
        p.node.setPosition(0, this.vh * 0.14);
        this.toggleRow(p.node, 165, 'icon_music', '背景音乐', 'music');
        this.toggleRow(p.node, 55, 'icon_sound', '游戏音效', 'sfx');
        this.toggleRow(p.node, -55, 'icon_vibrate', '震动反馈', 'vibrate');
        this.toggleRow(p.node, -165, 'icon_chain', '性能模式', 'lite');
        mkLabel(s, '性能模式：减少同屏怪物与特效，适合较旧的手机', 20, C_GREY, 'center', 2).node.setPosition(0,
    this.vh * 0.14 - 265);
        const stats = mkLabel(s, '', 26, C_GREY, 'center', 3);
        stats.node.setPosition(0, -this.vh * 0.2);
        this.refreshers.push(() => {
            const d = Save.data;
            stats.string = `游戏次数  ${d.runs}\n胜利次数  ${d.wins}\n累计击败  ${d.totalKills}\n最长存活  ${thi
    s.fmtTime(d.bestTime)}`;
```

```
        });
        mkButton(s, '返回', 300, 88, () => this.setState('menu'), C_BTN_BLUE).node.setPosition(0, this.bottom +
    80);
    }
    private buildPause() {
        const s = this.screen('pause');
        mkLabel(s, '暂停', 72, C_WHITE, 'center', 5).node.setPosition(0, this.vh * 0.3);
        const build = mkNode('build', s);
        build.setPosition(0, this.vh * 0.19);
        this.refreshers.push(() => {
            if (this.state !== 'pause') return;
            build.destroyAllChildren();
            const items = [
                ...this.battle.weapons.map(w => ({ icon: w.icon, lv: w.evolved ? '★' : String(w.level) })),
                ...this.battle.passives.map(p => ({ icon: p.def.icon, lv: String(p.level) })),
                ...this.battle.blessings.map(b => ({ icon: b.icon, lv: '◆' })),
            ];
            items.forEach((it, i) => {
                const x = ((i % 6) - 2.5) * 80, y = -Math.floor(i / 6) * 80;
                mkSprite(build, it.icon, 3).node.setPosition(x, y);
                mkLabel(build, it.lv, 18, C_WHITE, 'center', 2).node.setPosition(x + 20, y - 22);
            });
        });
        const y0 = -this.vh * 0.02;
        mkButton(s, '继续游戏', 420, 96, () => this.setState('play'), C_BTN_GREEN, 36).node.setPosition(0, y0);
        const music = mkButton(s, '', 200, 80, () => { Save.data.music = !Save.data.music; Save.save(); Sound.re
    freshMusic(); this.refreshAll(); }, C_BTN_GREY, 26);
        music.node.setPosition(-110, y0 - 120);
        const sfx = mkButton(s, '', 200, 80, () => { Save.data.sfx = !Save.data.sfx; Save.save(); this.refreshAl
    l(); }, C_BTN_GREY, 26);
        sfx.node.setPosition(110, y0 - 120);
        this.refreshers.push(() => {
            music.label.string = '音乐: ' + (Save.data.music ? '开' : '关');
            sfx.label.string = '音效: ' + (Save.data.sfx ? '开' : '关');
        });
        mkButton(s, '放弃本局', 420, 88, () => this.endRun(false), new Color(120, 30, 40, 255), 30).node.setPosi
    tion(0, y0 - 240);
    }
    private buildLevelUp() {
        const s = this.screen('levelup');
        this.levelTitle = mkLabel(s, '升级!', 60, C_GOLD, 'center', 5);
        this.levelTitle.node.setPosition(0, this.vh * 0.27);
        for (let i = 0; i < 3; i++) {
            const p = mkPanel(s, 660, 150);
            p.node.setPosition(0, this.vh * 0.13 - i * 170);
            const icon = mkSprite(p.node, 'icon_whip', 4.5);
            icon.node.setPosition(-260, 0);
            const name = mkLabel(p.node, '', 32, C_GOLD, 'left', 3);
            name.node.setPosition(-200, 34);
            const tag = mkLabel(p.node, '', 24, new Color(120, 220, 255, 255), 'right', 3);
```

```
            tag.node.setPosition(310, 34);
            const desc = mkLabel(p.node, '', 22, C_WHITE, 'left', 2);
            desc.node.setPosition(-200, -24);
            const idx = i;
            p.node.on(Node.EventType.TOUCH_END, () => this.choose(idx));
            this.cards.push({ node: p.node, icon, name, tag, desc, border: p.border, fill: p.fill });
        }
        this.rerollBtn = mkButton(s, '重抽', 300, 80, () => this.reroll(), C_BTN_BLUE, 28);
        this.rerollBtn.node.setPosition(0, this.vh * 0.13 - 3 * 170 + 20);
        const adY = this.vh * 0.13 - 3 * 170 + 20;
        this.adRerollBtn = mkButton(s, '', 380, 80, () => this.watchAd('reroll', () => {
            this.adRerolls--;
            this.options = this.battle.buildOptions();
            this.fillCards();
        }), C_AD, 26);
        this.adRerollBtn.node.setPosition(0, adY);
        this.adBlessBtn = mkButton(s, '▶ 看广告 额外多选一张', 440, 80, () => this.watchAd('blessing', () => {
            this.adBlessUsed = true;
            this.battle.pendingBlessings++;
            this.adBlessBtn.node.active = false;
            this.showToast('选完这张后还能再选一张!');
        }), C_AD, 26);
        this.adBlessBtn.node.setPosition(0, adY);
    }
    private buildChest() {
        const s = this.screen('chest');
        mkLabel(s, '宝箱!', 64, C_GOLD, 'center', 5).node.setPosition(0, this.vh * 0.3);
        mkSprite(s, 'chest', 6).node.setPosition(0, this.vh * 0.18);
        this.chestIcon = mkSprite(s, 'icon_whip', 6);
        this.chestIcon.node.setPosition(0, this.vh * 0.04);
        this.chestTitle = mkLabel(s, '', 40, C_GOLD, 'center', 4);
        this.chestTitle.node.setPosition(0, -this.vh * 0.06);
        this.chestDesc = mkLabel(s, '', 26, C_WHITE, 'center', 3);
        this.chestDesc.node.setPosition(0, -this.vh * 0.11);
        this.chestGold = mkLabel(s, '', 30, C_GOLD, 'center', 3);
        this.chestGold.node.setPosition(0, -this.vh * 0.16);
        this.chestAdBtn = mkButton(s, '▶ 看广告 再开一次', 400, 76, () => this.watchAd('chest', () => {
            this.adChestUsed = true;
            this.chestAdBtn.node.active = false;
            const r = this.battle.openChest(true);
            this.showChestResult(r, '再开一次: ');
        }), C_AD, 26);
        this.chestAdBtn.node.setPosition(0, -this.vh * 0.215);
        mkButton(s, '确定', 300, 90, () => { this.afterReward(); if (this.state === 'play') this.battle.levelBla
    st(); }, C_BTN_GREEN, 34).node.setPosition(0, -this.vh * 0.29);
    }
    private showChestResult(r: { icon: string; title: string; desc: string; evolved: boolean; gold: number }, pr
    efix = '') {
        this.chestIcon.spriteFrame = Art.f(r.icon);
        this.chestTitle.string = prefix + r.title;
```

```
        this.chestDesc.string = r.desc;
        this.chestGold.string = `+${r.gold} 金币`;
        Sound.play(r.evolved ? 'evolve' : 'chest');
    }
    private buildResult() {
        const s = this.screen('result');
        this.resultTitle = mkLabel(s, '', 76, C_RED, 'center', 6);
        this.resultTitle.node.setPosition(0, this.top - 110);
        this.resultStats = mkLabel(s, '', 28, C_WHITE, 'center', 3);
        this.resultStats.node.setPosition(0, this.top - 290);
        this.resultList = mkNode('list', s);
        this.resultHint = mkLabel(s, '', 24, new Color(255, 200, 120, 255), 'center', 3);
        this.resultHint.node.setPosition(0, this.top - 430);
        this.resultList.setPosition(0, this.top - 520);
        const by = this.bottom + 80;
        mkButton(s, '主菜单', 260, 88, () => { Ads.afterRun(this.adWatchedThisResult); this.toMenu(); }, C_BTN_G
    REY).node.setPosition(-150, by);
        mkButton(s, '再来一局', 260, 88, () => { Ads.afterRun(this.adWatchedThisResult); this.replay(); }, C_BTN
    _GREEN).node.setPosition(150, by);
        this.goldAdBtn = mkButton(s, '', 380, 80, () => this.watchAd('gold', () => {
            this.adGoldUsed = true;
            Save.data.gold += this.lastRunGold;
            Save.save();
            this.showToast(`金币 +${this.lastRunGold}!`);
            Sound.play('coin');
            this.refreshAll();
        }), C_AD, 26);
        this.shareBtn = mkButton(s, '分享战绩', 200, 80, () => this.shareResult(), C_BTN_BLUE, 26);
        // 抖音：分享本局录屏（按平台规则不发奖励）
        this.recBtn = mkButton(s, '分享本局录屏', 300, 72, () => {
            const b = this.battle;
            Recorder.share(b.won ? '我的奶蛙活过了 15 分钟! ' : `我的奶蛙坚持了 ${this.fmtTime(b.time)}，割倒 ${
    b.kills} 只怪物`, ok => { if (!ok) this.showToast('分享未完成'); });
        }, new Color(200, 40, 90, 255), 26);
        this.recBtn.node.setPosition(0, by + 186);
        this.refreshers.push(() => {
            if (this.state !== 'result') return;
            this.recBtn.node.active = Recorder.ready;
            const showAd = !this.adGoldUsed && this.lastRunGold > 0 && Ads.canReward('gold');
            const showShare = !!mgApi();
            this.goldAdBtn.node.active = showAd;
            this.goldAdBtn.label.string = `▶ 看广告 金币×2 (+${this.lastRunGold})`;
            this.shareBtn.node.active = showShare;
            this.goldAdBtn.node.setPosition(showShare ? -95 : 0, by + 100);
            this.shareBtn.node.setPosition(showAd ? 200 : 0, by + 100);
        });
    }
    // 死亡复活（每局一次，看广告满血复活）
    private buildRevive() {
        const s = this.screen('revive');
```

```
    mkLabel(s, '你倒下了…', 64, C_RED, 'center', 5).node.setPosition(0, this.vh * 0.2);
    mkLabel(s, '看一段广告，满状态原地复活?', 30, C_WHITE, 'center', 3).node.setPosition(0, this.vh * 0.1);
    this.reviveCount = mkLabel(s, '', 72, C_GOLD, 'center', 5, true);
    this.reviveCount.node.setPosition(0, 0);
    mkButton(s, '▶ 看广告复活', 420, 100, () => this.watchAd('revive', () => {
        this.adRevived = true;
        this.battle.revive(1);
        this.setState('play');
    }, () => { this.reviveT = Math.max(this.reviveT, 3); }), C_AD, 36).node.setPosition(0, -this.vh * 0.1);
    mkButton(s, '放弃', 260, 80, () => this.endRun(false), C_BTN_GREY, 28).node.setPosition(0, -this.vh * 0.
2);
}
// 启动页：动态夜景 + Logo 弹出 + 加载条 + 健康游戏忠告 + 适龄提示（法规要求）
private buildSplash() {
    const s = this.screen('splash');
    const horizon = -this.vh * 0.05;
    this.splashBackdrop = new Backdrop(s, this.vw, this.vh, horizon, this.vh * 0.2);
    this.splashLogo = mkNode('logo', s);
    this.splashLogo.setPosition(0, this.vh * 0.21);
    mkLabel(this.splashLogo, GAME_NAME, 104, new Color(60, 6, 16, 255), 'center', 0).node.setPosition(6, -8)
;
    mkLabel(this.splashLogo, GAME_NAME, 104, new Color(255, 236, 120, 255), 'center', 8);
    mkLabel(this.splashLogo, GAME_SUBTITLE, 28, new Color(255, 120, 110, 255), 'center', 4).node.setPosition
(0, -84);
    const heroH = Art.f('hero_hunter_0').rect.height;
    const hs = 2.4;
    this.splashHeroBaseY = horizon - 6 + (heroH * hs) / 2;
    mkSprite(s, 'shadow', hs * 1.7).node.setPosition(0, horizon - 4);
    this.splashHero = mkSprite(s, 'hero_hunter_0', hs);
    this.splashHero.node.setPosition(0, this.splashHeroBaseY);
    // 加载条
    const barY = -this.vh * 0.12;
    this.splashLoad = mkLabel(s, '', 22, new Color(230, 220, 240, 255), 'center', 2, true);
    this.splashLoad.node.setPosition(0, barY + 32);
    mkRect(s, 448, 22, new Color(20, 10, 30, 255), 'barBg').node.setPosition(0, barY);
    this.splashFill = mkRect(s, 440, 14, new Color(255, 214, 90, 255), 'barFill', 0, 0.5).node;
    this.splashFill.setPosition(-220, barY);
    mkLabel(s, '健康游戏忠告', 26, C_WHITE, 'center', 2).node.setPosition(0, -this.vh * 0.2);
    mkLabel(s, HEALTH_TIP, 21, new Color(200, 190, 210, 255), 'center', 2).node.setPosition(0, -this.vh * 0.
285);
    this.ageBadge(s, -this.vw / 2 + 56, this.top - 48);
    s.addComponent(UITransform).setContentSize(this.vw, this.vh);
    s.on(Node.EventType.TOUCH_END, () => { if (this.splashT > 1) this.afterSplash(); });
    this.refreshers.push(() => { this.splashHero.spriteFrame = Art.f(this.selectedChar().sprite + '_0'); });
}
private updateSplashFx(dt: number) {
    this.splashBackdrop.update(dt);
    const t = this.splashT;
    // Logo 弹出（easeOutBack）
    const k = Math.min(1, t / 0.6);
```

```
        const e = 1 + 2.7 * Math.pow(k - 1, 3) + 1.7 * Math.pow(k - 1, 2);
        const sc = Math.max(0.01, e) * (1 + Math.sin(this.t * 2) * 0.01);
        this.splashLogo.setScale(sc, sc, 1);
        // 奶蛙一蹦一蹦
        const hop = Math.abs(Math.sin(t * 4.5)) * 26;
        this.splashHero.node.setPosition(0, this.splashHeroBaseY + hop);
        this.splashHero.spriteFrame = Art.f(this.selectedChar().sprite + (hop > 8 ? '_2' : '_1'));
        const p = Math.min(1, t / 2.3);
        this.splashFill.setScale(p, 1, 1);
        this.splashLoad.string = `正在召唤奶蛙… ${Math.floor(p * 100)}%`;
    }
    private afterSplash() {
        if (this.state !== 'splash') return;
        this.setState(Save.data.privacyOk ? 'menu' : 'privacy');
    }
    // 首次启动：用户协议与隐私政策
    private buildPrivacy() {
        const s = this.screen('privacy');
        const p = mkPanel(s, 640, 620);
        p.node.setPosition(0, this.vh * 0.04);
        mkLabel(p.node, '用户协议与隐私政策', 34, C_GOLD, 'center', 3).node.setPosition(0, 250);
        mkLabel(p.node, `欢迎来到《${GAME_NAME}》！\n\n我们非常重视你的个人信息保护：\n• 本游戏不收集你的个人信息；\n• 游戏进度只保存在你的手机本地；\n• 游戏中的广告由微信平台提供，\n  相关信息处理遵循《微信隐私保护指引》；\n• 你可以随时在微信中删除本小游戏及其数据。\n\n点击“同意”即表示你已阅读并同意\n《用户协议》和《隐私政策》。`, 22, C_WHITE, 'center', 2).node.setPosition(0, 10);
        mkButton(p.node, '不同意', 220, 80, () => {
            const w = mgApi();
            if (w && w.exitMiniProgram) w.exitMiniProgram({});
            else this.showToast('需要同意后才能开始游戏');
        }, C_BTN_GREY, 28).node.setPosition(-130, -250);
        mkButton(p.node, '同意', 220, 80, () => {
            Save.data.privacyOk = true;
            Save.save();
            this.setState('menu');
        }, C_BTN_GREEN, 30).node.setPosition(130, -250);
    }
    // 看激励视频：看完发奖；没看完或失败给出提示
    private watchAd(place: string, onReward: () => void, onFail?: () => void) {
        Ads.showReward(place, (r: RewardResult) => {
            if (r.ok) {
                this.adWatchedThisResult = true;
                onReward();
            } else {
                this.showToast(r.reason === 'skipped' ? '完整观看广告才能获得奖励' : '广告暂时不可用，请稍后再试');
                if (onFail) onFail();
            }
            this.refreshAll();
        });
    }
```

```
    // ------------------------------------------------------------ 状态
    private setState(s: State) {
        // 每天第一次回到主菜单时自动弹出签到（新玩家第一局之前不弹）
        if (s === 'menu' && this.signDue() && Save.data.signShown !== this.today && Save.data.runs > 0) {
            Save.data.signShown = this.today;
            s = 'signin';
        }
        const prev = this.state;
        this.state = s;
        for (const k in this.screens) this.screens[k as State]!.active = k === s;
        const inRun = s === 'play' || s === 'levelup' || s === 'chest' || s === 'pause' || s === 'revive';
        this.hud.active = inRun;
        this.overlay.node.active = s !== 'play';
        this.overlay.color = inRun || s === 'result' ? C_DIM : new Color(0, 0, 0, s === 'menu' ? 90 : 190);
        this.touchLayer.active = s === 'play';
        if (s !== 'play') this.onJoyEnd();
        if (s === 'menu' || s === 'chars' || s === 'shop' || s === 'settings' || s === 'achv' || s === 'codex' |
| s === 'signin' || s === 'privacy' || s === 'daily') Sound.playBgm('bgm_menu');
        if (s === 'codex') { Save.data.codexSeen = Save.data.codex.length; Save.save(); }
        if (s === 'revive') { this.reviveT = 8; Sound.pauseBgm(); }
        if (prev === 'revive' && s === 'play') Sound.resumeBgm();
        if (s === 'pause') { Sound.pauseBgm(); Recorder.pause(); }
        if (prev === 'pause' && s === 'play') Recorder.resume();
        if (prev === 'pause' && s === 'play') Sound.resumeBgm();
        this.refreshAll();
    }
    private refreshAll() {
        for (const f of this.refreshers) f();
    }
    // 活动地图只在活动期间开放；普通地图按胜利依次解锁
    private stageOpen(id: string) {
        const st = STAGES.find(x => x.id === id);
        if (!st) return false;
        if (st.event) { const ev = activeEvent(); return !!ev && ev.id === st.event; }
        return Save.data.stages.includes(id);
    }
    private get currentStage() { return STAGES.find(x => x.id === Save.data.stage && this.stageOpen(x.id)) || ST
AGES[0]; }
    private get endlessMode() { return Save.data.mode === 'endless' && Save.data.wins > 0; }
    private get nightmareMode() { return Save.data.mode === 'nightmare' && Save.data.wins > 0; }
    private toMenu() {
        this.dailyRun = false;
        this.battle.reset(this.selectedChar());
        this.battle.setPlayerVisible(false);
        this.setState('menu');
    }
    private get today() { return new Date().toDateString(); }
    // 每日挑战：固定试玩角色、地图、规则与命运卡牌，存活 10 分钟
    private startDaily() {
        const ds = dailySetup(this.today);
```

```
        const ch = CHARACTERS.find(c => c.id === ds.char) || CHARACTERS[0];
        this.dailyRun = true;
        this.tutStep = -1;
        this.adRevived = this.adBlessUsed = false;
        this.adRerolls = AD_CONFIG.rerollPerRun;
        this.battle.reset(ch, false, ds.stage, { mod: ds.mod, blessing: ds.blessing });
        this.battle.nightmare = false;
        this.battle.setPlayerVisible(true);
        this.refreshIcons();
        this.bannerT = 0;
        Sound.play('start');
        Sound.playBgm('bgm_battle');
        Recorder.start();
        this.setState('play');
        this.banner(`每日挑战 · ${ds.mod.name}`, new Color(255, 170, 80, 255));
    }
    private replay() {
        if (this.dailyRun) this.startDaily();
        else this.startGame();
    }
    private startGame() {
        this.dailyRun = false;
        this.adRevived = this.adBlessUsed = false;
        this.adRerolls = AD_CONFIG.rerollPerRun;
        this.battle.reset(this.selectedChar(), this.endlessMode, this.currentStage.id);
        this.battle.nightmare = this.nightmareMode;
        this.battle.setPlayerVisible(true);
        this.refreshIcons();
        this.bannerT = 0;
        this.bannerLabel.node.active = false;
        Sound.play('start');
        Sound.playBgm('bgm_battle');
        Recorder.start();
        this.setState('play');
        this.tutStart();
        const st = this.battle.streak;
        this.banner(this.endlessMode ? '无尽之夜 · 能撑多久?' : this.nightmareMode ? `噩梦难度 · ${this.curren
    tStage.name}` : st > 0 ? `不屈意志 · 伤害与生命 +${st * 5}%` : `${this.currentStage.name} · 生存下去!`, ne
    w Color(255, 230, 120, 255));
    }
    // 结算：发放金币、记录最佳成绩
    private endRun(win: boolean) {
        Recorder.stop();
        if (this.tutStep >= 0) this.tutFinish();
        this.tutNode.active = false;
        const b = this.battle;
        const d = Save.data;
        const bonus = Math.round((Math.floor(b.time / 60) * 25 + (win ? 500 : 0)) * b.stats.greed * b.stage.gold
    Mul * (b.nightmare ? 1.6 : 1));
        let total = b.gold + bonus;
```

```
    // 每日首局金币 ×2
    const today = new Date().toDateString();
    const daily = d.bonusDay !== today;
    if (daily) { d.bonusDay = today; total *= 2; }
    d.gold += total;
    this.lastRunGold = total;
    this.adGoldUsed = false;
    this.adWatchedThisResult = false;
    // 每日挑战：首次成功发奖励，不计入连败与地图解锁
    this.dailyReward = 0;
    if (this.dailyRun) {
        if (d.dailyBestDate !== this.today) { d.dailyBestDate = this.today; d.dailyBest = 0; }
        d.dailyBest = Math.max(d.dailyBest, Math.floor(b.time));
        if (win && d.dailyDone !== this.today) {
            d.dailyDone = this.today;
            this.dailyReward = DAILY_REWARD;
            d.gold += DAILY_REWARD;
        }
    }
    if (!b.endless && !this.dailyRun) d.lossStreak = win ? 0 : Math.min(3, (d.lossStreak || 0) + 1);
    // 地图：记录最佳，胜利后解锁下一张
    d.stageBest[b.stage.id] = Math.max(d.stageBest[b.stage.id] || 0, Math.floor(b.time));
    this.newStage = '';
    // 限时活动：击败限定 Boss 永久解锁限定角色
    this.newChar = '';
    for (const ev of EVENTS) {
        if (b.bossKills.includes(ev.boss) && !d.chars.includes(ev.char)) {
            d.chars.push(ev.char);
            this.newChar = (CHARACTERS.find(c => c.id === ev.char) || { name: '' }).name;
        }
    }
    if (win && !b.endless && !this.dailyRun) {
        const i = STAGES.findIndex(x => x.id === b.stage.id);
        const next = STAGES[i + 1];
        if (next && !d.stages.includes(next.id)) { d.stages.push(next.id); this.newStage = next.name; }
    }
    d.runs++;
    if (win) d.wins++;
    d.totalKills += b.kills;
    d.bestTime = Math.max(d.bestTime, Math.floor(b.time));
    d.bestKills = Math.max(d.bestKills, b.kills);
    d.bestLevel = Math.max(d.bestLevel, b.level);
    if (b.endless) d.endlessBest = Math.max(d.endlessBest, Math.floor(b.time));
    // 每日任务（跨局累计，完成即发金币）
    const qr = this.updateQuests();
    d.gold += qr.gold;
    this.questMsg = qr.done.length ? `每日任务完成 ${qr.done.length} 个${qr.bonus ? '，今日任务全部完成！ ' :
''}  +${qr.gold} 金币` : '';
    // 成就结算
    const got = this.checkAchievements(win);
```

```
        let achGold = 0;
        for (const a of got) achGold += a.gold;
        d.gold += achGold;
        Save.save();
        this.resultTitle.string = this.dailyRun ? (win ? '挑战成功!' : '挑战失败') : win ? '胜利!' : b.endless ?
 '无尽之夜结束' : '你倒下了';
        this.resultTitle.color = win ? C_GOLD : C_RED;
        this.resultStats.string = `存活时间  ${this.fmtTime(b.time)}\n等级  ${b.level}     击败  ${b.kills}
最高连击  ${b.maxCombo}\n拾取金币  ${b.gold}    时间奖励  ${bonus}\n本局获得  ${total + achGold} 金币` + (go
t.length ? `\n新成就: ${got.map(a => a.name).join('、')}  (+${achGold})` : '');
        this.resultHint.string = this.nearMiss(win, daily);
        // 按行数依次排版: 统计 → 提示 → 武器伤害列表
        const statLines = this.resultStats.string.split('\n').length, hintLines = this.resultHint.string ? this.
resultHint.string.split('\n').length : 0;
        const statTop = this.top - 200;
        this.resultStats.node.setPosition(0, statTop - (statLines * 34) / 2);
        const hintTop = statTop - statLines * 34 - 14;
        this.resultHint.node.setPosition(0, hintTop - (hintLines * 30) / 2);
        this.resultList.setPosition(0, hintTop - hintLines * 30 - 50);
        this.resultList.destroyAllChildren();
        mkLabel(this.resultList, '武器伤害统计', 26, C_GOLD, 'center', 3).node.setPosition(0, 30);
        b.weapons.forEach((w, i) => {
            const y = -24 - i * 58;
            mkSprite(this.resultList, w.icon, 2.5).node.setPosition(-230, y);
            mkLabel(this.resultList, w.name + (w.evolved ? '' : `  LV ${w.level}`), 24, C_WHITE, 'left', 2).node
.setPosition(-195, y);
            mkLabel(this.resultList, String(Math.round(w.damage)), 24, C_GOLD, 'right', 2).node.setPosition(250,
 y);
        });
        Sound.playBgm('');
        Sound.play(win ? 'win' : 'gameover');
        if (got.length) this.scheduleOnce(() => Sound.play('achieve'), 1.2);
        vibrate(true);
        this.setState('result');
    }
    private questMsg = '';
    // 今日任务进度（日期变化时清零）
    private questProgress() {
        const d = Save.data;
        if (d.questDate !== this.today) { d.questDate = this.today; d.questProg = {}; d.questDone = []; }
        return d.questProg;
    }
    private updateQuests(): { done: QuestDef[]; bonus: boolean; gold: number } {
        const b = this.battle, d = Save.data;
        const p = this.questProgress();
        const add = (k: string, v: number) => { p[k] = (p[k] || 0) + v; };
        const best = (k: string, v: number) => { p[k] = Math.max(p[k] || 0, v); };
        add('kills', b.kills);
        add('boss', b.bossKills.length);
        add('evolve', b.evolvedCount);
```

```
        add('gold', b.gold);
        add('chest', b.chestsOpened);
        add('runs', 1);
        best('time', Math.floor(b.time));
        best('level', b.level);
        best('combo', b.maxCombo);
        const qs = dailyQuests(this.today);
        const done: QuestDef[] = [];
        let gold = 0;
        for (const q of qs) {
            if (d.questDone.includes(q.id) || (p[q.stat] || 0) < q.n) continue;
            d.questDone.push(q.id);
            done.push(q);
            gold += q.gold;
        }
        let bonus = false;
        if (qs.every(q => d.questDone.includes(q.id)) && !d.questDone.includes('bonus')) {
            d.questDone.push('bonus');
            bonus = true;
            gold += QUEST_BONUS;
        }
        return { done, bonus, gold };
    }
    private codexCount() {
        const c = Save.data.codex;
        return codexEntries().filter(e => c.includes(e.key)).length;
    }
    // 结算提示：差一点的遗憾感 + 下一个可达成的目标，驱动"再来一局"
    private nearMiss(win: boolean, daily: boolean) {
        const b = this.battle, d = Save.data;
        const lines: string[] = [];
        if (this.dailyReward) lines.push(`每日挑战奖励：+${this.dailyReward} 金币！`);
        if (this.questMsg) lines.push(this.questMsg);
        if (this.newChar) lines.push(`解锁限定角色：${this.newChar}！`);
        if (this.newStage) lines.push(`解锁新地图：${this.newStage}！`);
        if (b.newCodex.length) lines.push(`图鉴新收录 ${b.newCodex.length} 项（${this.codexCount()}/${codexEntri
es().length}）`);
        if (daily) lines.push('每日首局奖励：金币 ×2！');
        if (!win && !b.endless) {
            if (b.boss && b.boss.alive) lines.push(`${b.boss.def.name} 只剩 ${Math.max(1, Math.round((b.boss.hp
/ b.boss.maxHp) * 100))}% 生命了！`);
            else {
                const nb = BOSS_SCHEDULE.find(x => x.t > b.time);
                const goal = this.dailyRun ? DAILY_GOAL : WIN_TIME;
                const left = Math.ceil((nb && nb.t < goal ? nb.t : goal) - b.time);
                lines.push(nb && nb.t < goal ? `再坚持 ${this.fmtTime(left)} 就能见到 ${ENEMIES[nb.id].name}` :
`距离胜利只差 ${this.fmtTime(left)}！`);
            }
            if (d.lossStreak > 0) lines.push(`下一局获得「不屈意志」伤害与生命 +${d.lossStreak * 5}%`);
        }
```

```
        // 下一个强化目标
        let best: { name: string; cost: number } | null = null;
        for (const p of POWERUPS) {
            const r = Save.rank(p.id);
            if (r >= p.max) continue;
            const c = powerUpCost(p, r, Save.totalBought());
            if (!best || c < best.cost) best = { name: `${p.name} Lv${r + 1}`, cost: c };
        }
        if (best) lines.push(d.gold >= best.cost ? `可以购买强化：${best.name}!` : `下一个强化：${best.name}（还差 ${best.cost - d.gold} 金币）`);
        return lines.join('\n');
    }
    // 结算时检查成就，返回本局新解锁的成就
    private checkAchievements(win: boolean) {
        const b = this.battle, d = Save.data;
        const ok: Record<string, boolean> = {
            win1: win,
            kill1k: b.kills >= 1000,
            kill5k: b.kills >= 5000,
            combo500: b.maxCombo >= 500,
            lv50: b.level >= 50,
            evo1: b.evolvedCount >= 1,
            evo3: b.evolvedCount >= 3,
            goldrat: b.goldRats > 0,
            endless20: b.endless && b.time >= 1200,
            allchars: CHARACTERS.every(c => c.cost < 0 || d.chars.includes(c.id)),
            daily1: this.dailyRun && win,
            stage_snow: win && !b.endless && !this.dailyRun && b.stage.id === 'snow',
            stage_lava: win && !b.endless && !this.dailyRun && b.stage.id === 'lava',
            stage_swamp: win && !b.endless && !this.dailyRun && b.stage.id === 'swamp',
            nightmare1: win && b.nightmare,
            codex_half: this.codexCount() * 2 >= codexEntries().length,
            codex_full: this.codexCount() >= codexEntries().length,
        };
        for (const id of b.bossKills) ok['boss_' + id] = true;
        const got = ACHIEVEMENTS.filter(a => ok[a.id] && !d.achievements.includes(a.id));
        for (const a of got) d.achievements.push(a.id);
        return got;
    }
    // ---------------------------------------------------------------- 七日签到
    private signDue() { return Save.data.signDate !== this.today; }
    private claimSignin(mul: number) {
        const d = Save.data;
        if (!this.signDue()) return;
        const day = d.signCount % 7;
        const g = SIGNIN_REWARDS[day] * mul;
        d.gold += g;
        d.signCount++;
        d.signDate = this.today;
        Save.save();
```

```
        Sound.play(day === 6 ? 'achieve' : 'coin');
        this.showToast(`签到成功 +${g} 金币` + (mul > 1 ? '（双倍）' : ''));
        this.refreshAll();
    }
    private buildSignin() {
        const s = this.screen('signin');
        const p = mkPanel(s, 660, 760, new Color(30, 22, 52, 250), C_GOLD);
        p.node.setPosition(0, this.vh * 0.03);
        mkLabel(p.node, '七日签到', 46, C_GOLD, 'center', 4).node.setPosition(0, 330);
        const sub = mkLabel(p.node, '', 22, C_GREY, 'center', 2);
        sub.node.setPosition(0, 280);
        // 7 个格子：上排 4 个，下排 3 个（第 7 天大格子）
        const cells: { panel: { fill: Sprite; border: Sprite }; check: Node; amount: Label }[] = [];
        for (let i = 0; i < 7; i++) {
            const big = i === 6;
            const w = big ? 290 : 140, h = 170;
            const y = i < 4 ? 150 : -40;
            const c = mkPanel(p.node, w, h);
            c.node.setPosition(i < 4 ? (i - 1.5) * 152 : [-228, -76, 152][i - 4], y);
            mkLabel(c.node, `第 ${i + 1} 天`, 22, C_WHITE, 'center', 2).node.setPosition(0, 60);
            mkSprite(c.node, big ? 'chest' : 'coin', big ? 2.4 : 3).node.setPosition(0, 6);
            const amount = mkLabel(c.node, `${SIGNIN_REWARDS[i]}`, 26, C_GOLD, 'center', 3);
            amount.node.setPosition(0, -56);
            const check = mkLabel(c.node, '已领', 30, C_GREEN, 'center', 4).node;
            check.setPosition(0, 6);
            cells.push({ panel: c, check, amount });
        }
        const claim = mkButton(p.node, '领取', 260, 88, () => { this.claimSignin(1); this.setState('menu'); }, C
_BTN_GREEN, 34);
        claim.node.setPosition(-140, -240);
        const dbl = mkButton(p.node, '▶ 看视频 双倍', 260, 88, () => this.watchAd('signin', () => { this.claimS
ignin(2); this.setState('menu'); }), new Color(170, 90, 30, 255), 28);
        dbl.node.setPosition(140, -240);
        const close = mkButton(p.node, '关闭', 200, 64, () => this.setState('menu'), C_BTN_GREY, 26);
        close.node.setPosition(0, -330);
        this.refreshers.push(() => {
            const d = Save.data;
            const due = this.signDue();
            // 本轮进度：今天已签则显示到今天为止
            const done = due ? d.signCount % 7 : ((d.signCount - 1) % 7) + 1;
            const today = due ? d.signCount % 7 : -1;
            sub.string = due ? `累计签到 ${d.signCount} 天 · 断签不清零` : `今天已签到，明天再来呱～（累计 ${d.
signCount} 天）`;
            cells.forEach((c, i) => {
                const got = i < done;
                c.check.active = got;
                c.panel.border.color = i === today ? C_GREEN : got ? new Color(90, 80, 110, 255) : i === 6 ? C_G
OLD : C_BORDER;
                c.panel.fill.color = got ? new Color(24, 20, 34, 250) : i === today ? new Color(36, 64, 60, 250)
 : new Color(36, 30, 64, 245);
```

```
        });
        claim.node.active = due;
        dbl.node.active = due && Ads.canReward('signin');
        if (!dbl.node.active) claim.node.setPosition(0, -240); else claim.node.setPosition(-140, -240);
    });
}
// ---------------------------------------------------------------- 图鉴
private codexTab: CodexTab = 'weapon';
private codexSel: CodexEntry | null = null;
private buildCodex() {
    const s = this.screen('codex');
    this.header(s, '图鉴', false);
    const all = codexEntries();
    const prog = mkLabel(s, '', 26, C_GREY, 'center', 2);
    prog.node.setPosition(0, this.top - 118);
    // 分页按钮
    const tabs: [CodexTab, string][] = [['weapon', '武器'], ['passive', '道具'], ['monster', '怪物']];
    const tabBtns: Record<string, Button> = {};
    tabs.forEach(([id, label], i) => {
        const b = mkButton(s, label, 200, 70, () => { this.codexTab = id; this.codexSel = null; this.refresh
All(); }, C_BTN_BLUE, 30);
        b.node.setPosition((i - 1) * 216, this.top - 180);
        tabBtns[id] = b;
    });
    // 详情面板（底部）
    const detH = 190;
    const det = mkPanel(s, this.vw - 40, detH);
    det.node.setPosition(0, this.bottom + 170 + detH / 2);
    const dIcon = mkSprite(det.node, 'icon_whip', 3);
    dIcon.node.setPosition(-this.vw / 2 + 90, 10);
    const dName = mkLabel(det.node, '', 32, C_GOLD, 'left', 3);
    dName.node.setPosition(-this.vw / 2 + 160, 52);
    const dDesc = mkLabel(det.node, '', 22, C_WHITE, 'left', 2);
    dDesc.node.setPosition(-this.vw / 2 + 160, -20);
    // 图标网格：每页单独一组节点
    const gridTop = this.top - 240, gridBottom = this.bottom + 190 + detH;
    for (const [tab] of tabs) {
        const list = all.filter(e => e.tab === tab);
        const grid = mkNode('grid_' + tab, s);
        const cols = tab === 'weapon' ? 4 : 5; // 武器页：每行两组「武器 → 进化」
        const rows = Math.ceil(list.length / cols);
        const cell = Math.min(124, (gridTop - gridBottom) / rows - 8, (this.vw - 60) / cols - 8);
        list.forEach((e, i) => {
            const c = i % cols, r = Math.floor(i / cols);
            const card = mkPanel(grid, cell, cell);
            card.node.setPosition((c - (cols - 1) / 2) * (cell + 10), gridTop - cell / 2 - r * (cell + 8));
            const isBoss = tab === 'monster' && !!ENEMIES[e.key.slice(2)].boss;
            // 图标按格子大小等比缩放（Boss 精灵较大）
            const fr = Art.f(e.icon).rect;
            const fit = Math.min(3, (cell - 22) / Math.max(fr.width, fr.height));
```

```
            const sp = mkSprite(card.node, e.icon, fit);
            card.node.on(Node.EventType.TOUCH_END, () => { Sound.play('click'); this.codexSel = e; this.refr
eshAll(); });
            this.refreshers.push(() => {
                const got = Save.data.codex.includes(e.key);
                sp.color = got ? new Color(255, 255, 255, 255) : new Color(0, 0, 0, 160);
                const sel = this.codexSel === e;
                card.border.color = sel ? C_GOLD : isBoss ? new Color(170, 60, 90, 255) : e.key[0] === 'e' ?
 new Color(220, 110, 60, 255) : new Color(90, 80, 110, 255);
                card.fill.color = got ? new Color(44, 36, 72, 250) : new Color(22, 18, 32, 245);
            });
        });
        this.refreshers.push(() => { grid.active = this.codexTab === tab; });
    }
    this.refreshers.push(() => {
        const d = Save.data;
        const n = all.filter(e => d.codex.includes(e.key)).length;
        prog.string = `已收录 ${n} / ${all.length}`;
        for (const [id] of tabs) tabBtns[id].node.setScale(this.codexTab === id ? 1.06 : 0.94, this.codexTab
 === id ? 1.06 : 0.94, 1);
        const e = this.codexSel;
        det.node.active = !!e;
        if (!e) return;
        const got = d.codex.includes(e.key);
        dIcon.spriteFrame = Art.f(e.icon);
        const r = Art.f(e.icon).rect, k = Math.min(5, 150 / Math.max(r.width, r.height));
        dIcon.node.setScale(k, k, 1);
        dIcon.color = got ? new Color(255, 255, 255, 255) : new Color(0, 0, 0, 180);
        dName.string = got ? e.name : '? ? ? ';
        dDesc.string = got ? e.desc : '未收录: ' + e.hint;
    });
    mkButton(s, '返回', 300, 88, () => this.setState('menu'), C_BTN_BLUE).node.setPosition(0, this.bottom +
80);
}
private buildDaily() {
    const s = this.screen('daily');
    this.header(s, '每日挑战', false);
    const date = mkLabel(s, '', 26, C_GREY, 'center', 2);
    date.node.setPosition(0, this.top - 118);
    // 试玩角色
    const heroY = this.top - 300;
    const hero = mkSprite(s, 'hero_hunter_0', 2.6);
    hero.node.setPosition(-160, heroY);
    const heroName = mkLabel(s, '', 30, C_WHITE, 'left', 3);
    heroName.node.setPosition(-40, heroY + 50);
    const heroTag = mkLabel(s, '今日试玩角色（无需解锁）', 20, C_GREEN, 'left', 2);
    heroTag.node.setPosition(-40, heroY + 8);
    const stageName = mkLabel(s, '', 22, new Color(180, 220, 255, 255), 'left', 2);
    stageName.node.setPosition(-40, heroY - 32);
    // 规则卡 + 命运卡
```

```
    c.poly([[24, 12], [34, 18], [38, 40], [10, 40], [14, 18]], ROBE);
    c.poly([[14, 18], [10, 40], [17, 40], [18, 22]], ROBED);
    for (let x = 10; x < 38; x += 4) c.rect(x + f * 2, 40, 2, 3 + ((x >> 2) % 2), ROBE);
    c.ellipse(24, 15, 5.5, 5, SKIN);
    c.rect(21, 13, 2, 2, OUT); c.rect(26, 13, 2, 2, OUT);
    c.poly([[18, 16], [30, 16], [27, 28], [24, 31], [21, 28]], 0xf4f8ff);
    c.rect(20, 11, 4, 1, 0xf4f8ff); c.rect(25, 11, 4, 1, 0xf4f8ff);
    c.poly([[14, 11], [34, 11], [26, 0], [24, -1]], ROBE);
    c.rect(14, 10, 20, 2, ROBED);
    c.px(24, 4, 0xffe060); c.px(21, 7, 0xffe060); c.px(27, 8, 0xffe060);
    c.rect(36, 23, 4, 3, SKIN);
    c.outline(OUT);
    return c;
}
function drawEye(f: number) {
    // 水晶魔眼：紫色水晶框里的魔法宝石眼，周围漂浮水晶碎片
    const c = new PixelCanvas(48, 48);
    const CRY = 0x8a5ad0, CRYD = 0x5a3a9a, CRYL = 0xc8a8ff;
    for (let k = 0; k < 5; k++) {
        const x0 = 12 + k * 6, y0 = 38 + ((k + f) % 2) * 3;
        c.poly([[x0, y0 - 4], [x0 + 3, y0], [x0, y0 + 5], [x0 - 3, y0]], k % 2 ? CRYL : CRY);
    }
    c.circle(24, 21, 17, CRYD);
    c.circle(24, 21, 15, CRY);
    for (let k = 0; k < 8; k++) {
        const a = (k / 8) * Math.PI * 2;
        c.poly([[24 + Math.cos(a) * 14, 21 + Math.sin(a) * 14], [24 + Math.cos(a + 0.2) * 18, 21 + Math.sin(a +
    0.2) * 18], [24 + Math.cos(a + 0.4) * 14, 21 + Math.sin(a + 0.4) * 14]], CRYL);
    }
    c.circle(24, 22, 11.5, 0xf4f0ff);
    const ix = 24 + (f ? 2 : 0);
    c.circle(ix, 22, 7, 0x3a7ae0);
    c.circle(ix, 22, 5, 0x6ab8ff);
    c.circle(ix, 22, 2.5, 0x1a2a60);
    c.circle(ix - 3, 19, 2, 0xffffff);
    c.outline(OUT);
    return c;
}
function drawEnemyShot(kind: string) {
    switch (kind) {
        case 'bone': {
            // 铁甲将军的尖刺（钢刺）
            const c = new PixelCanvas(16, 8);
            c.poly([[0, 4], [4, 1], [16, 4], [4, 7]], 0xa0a8b8);
            c.poly([[3, 4], [5, 2], [14, 4]], 0xe0e6f0);
            c.outline(OUT);
            return c;
        }
        case 'blood': {
            const c = new PixelCanvas(12, 12);
```

```
            // 蝙蝠伯爵的紫色魔法弹
            c.circle(6, 6, 5.5, 0x4a1a80);
            c.circle(6, 6, 4.5, 0xa050e0);
            c.circle(4.5, 4.5, 1.5, 0xe8c0ff);
            return c;
        }
        case 'ice': {
            const c = new PixelCanvas(16, 8);
            c.poly([[0, 4], [5, 1], [16, 4], [5, 7]], 0x80e0ff);
            c.poly([[3, 4], [6, 2], [14, 4]], 0xe0f8ff);
            c.outline(0x1a3a5a);
            return c;
        }
        case 'darkorb': {
            const c = new PixelCanvas(14, 14);
            c.circle(7, 7, 6.5, 0x8a30c0, 120);
            c.circle(7, 7, 5, 0x6a1aa0);
            c.circle(7, 7, 3, 0xd080ff);
            c.px(6, 5, 0xffffff);
            return c;
        }
        case 'slimeball': {
            const c = new PixelCanvas(12, 12);
            c.circle(6, 6, 5, 0x2a8a3a);
            c.circle(6, 6, 4, 0x6fe06a);
            c.px(4, 4, 0xffffff);
            c.outline(OUT);
            return c;
        }
        case 'scythe': {
            // 暗夜魔术师的回旋飞牌
            const c = new PixelCanvas(28, 28);
            c.rect(7, 3, 14, 22, 0xf8f8f8);
            c.rect(9, 5, 10, 18, 0xe0e0ec);
            c.poly([[14, 9], [18, 14], [14, 19], [10, 14]], 0xd04a8a);
            c.px(9, 5, 0xd04a8a); c.px(18, 22, 0xd04a8a);
            c.outline(OUT);
            return c;
        }
    }
    return new PixelCanvas(4, 4);
}
function drawWarn() {
    const c = new PixelCanvas(64, 64);
    c.circle(32, 32, 31.5, 0xffffff, 70);
    c.ring(32, 32, 31.5, 3, 0xffffff, 230);
    return c;
}
// ------------------------------------------------------------- 武器 / 特效
function drawBolt() {
```

```
    const c = new PixelCanvas(14, 14);
    c.circle(7, 7, 6.5, 0x3fa9ff, 110);
    c.circle(7, 7, 4.5, 0x3fa9ff);
    c.circle(7, 7, 2.5, 0xbfe8ff);
    c.px(6, 6, 0xffffff);
    return c;
}
function drawKnife() {
    const c = new PixelCanvas(18, 8);
    c.rect(1, 3, 4, 2, 0x6b3a1f);
    c.rect(5, 1, 1, 6, 0xf5c542);
    c.rect(6, 3, 9, 2, 0xd8dde8);
    c.rect(6, 3, 9, 1, 0xffffff);
    c.px(15, 3, 0xd8dde8);
    c.px(16, 4, 0xb0b8c8);
    c.outline(OUT);
    return c;
}
function drawAxe() {
    const c = new PixelCanvas(18, 18);
    c.line(3, 15, 12, 6, 0x8a5a30, 255, 2);
    c.poly([[9, 2], [16, 4], [16, 11], [12, 9], [9, 6]], 0xb8c0cc);
    c.poly([[14, 3], [16, 4], [16, 11], [14, 10]], 0xe8eef4);
    c.line(9, 3, 9, 6, 0x7a8290);
    c.outline(OUT);
    return c;
}
function drawBible() {
    const c = new PixelCanvas(14, 16);
    c.rect(1, 1, 12, 14, 0x5a3aa0);
    c.rect(1, 1, 2, 14, 0x3a2470);
    c.rect(11, 2, 2, 12, 0xf4ecd8);
    c.poly([[7, 4], [8, 7], [11, 7], [8.5, 9], [9.5, 12], [7, 10], [4.5, 12], [5.5, 9], [3, 7], [6, 7]], 0xf5c54
    2);
    c.outline(OUT);
    return c;
}
function drawLightning() {
    const c = new PixelCanvas(20, 96);
    const r = rng(7);
    let x = 10;
    for (let y = 0; y < 96; y += 6) {
        const nx = Math.max(4, Math.min(15, x + Math.round((r() - 0.5) * 10)));
        c.line(x, y, nx, y + 6, 0xa0c8ff, 160, 5);
        x = nx;
    }
    x = 10;
    const r2 = rng(7);
    for (let y = 0; y < 96; y += 6) {
        const nx = Math.max(4, Math.min(15, x + Math.round((r2() - 0.5) * 10)));
```

```
        c.line(x, y, nx, y + 6, 0xffffff, 255, 2);
        x = nx;
    }
    return c;
}
function drawSpark() {
    const c = new PixelCanvas(24, 24);
    c.circle(12, 12, 7, 0xfff4a0, 140);
    c.poly([[12, 0], [14, 10], [24, 12], [14, 14], [12, 24], [10, 14], [0, 12], [10, 10]], 0xfff8c0);
    c.circle(12, 12, 3, 0xffffff);
    return c;
}
function drawSpike() {
    const c = new PixelCanvas(20, 26);
    c.poly([[2, 25], [6, 6], [9, 25]], 0x8a7a6a);
    c.poly([[7, 25], [11, 0], [15, 25]], 0xb8a890);
    c.poly([[13, 25], [17, 9], [19, 25]], 0x8a7a6a);
    c.line(11, 2, 11, 22, 0xf0e8d8);
    c.line(6, 9, 6, 22, 0xd0c4b0);
    c.outline(OUT);
    return c;
}
function drawSpikeWarn() {
    const c = new PixelCanvas(16, 8);
    c.ellipse(8, 4, 7, 3, 0x5a4030, 200);
    c.px(5, 3, 0x2a1a10); c.px(8, 4, 0x2a1a10); c.px(11, 3, 0x2a1a10);
    return c;
}
function drawWhip() {
    const c = new PixelCanvas(64, 24);
    c.ellipse(32, 22, 31.5, 18, 0xa8d0ff, 200);
    c.ellipse(32, 23, 29, 15, 0xffffff, 240);
    // 挖空形成月牙
    for (let y = 0; y < 24; y++) for (let x = 0; x < 64; x++) {
        const dx = (x + 0.5 - 32) / 27, dy = (y + 0.5 - 26) / 15;
        if (dx * dx + dy * dy <= 1) c.clear(x, y);
    }
    return c;
}
function drawAura() {
    const c = new PixelCanvas(64, 64);
    c.circle(32, 32, 31.5, 0xfff0c0, 40);
    c.ring(32, 32, 31.5, 2, 0xfff0c0, 120);
    for (let i = 0; i < 12; i++) {
        const a = (i / 12) * Math.PI * 2;
        c.rect(32 + Math.cos(a) * 26 - 1, 32 + Math.sin(a) * 26 - 1, 2, 2, 0xffffff, 120);
    }
    return c;
}
function drawGem(main: number, light: number, dark: number) {
```

```
    const c = new PixelCanvas(10, 12);
    c.poly([[5, 0], [10, 5], [5, 12], [0, 5]], main);
    c.poly([[5, 0], [5, 5], [0, 5]], light);
    c.poly([[5, 12], [10, 5], [5, 6]], dark);
    c.px(3, 3, 0xffffff);
    c.outline(OUT);
    return c;
}
function drawChicken() {
    const c = new PixelCanvas(18, 14);
    c.rect(1, 9, 6, 2, 0xf4ecd8);
    c.circle(1.5, 8.5, 1.5, 0xf4ecd8);
    c.circle(1.5, 11.5, 1.5, 0xf4ecd8);
    c.ellipse(11, 7, 6.5, 5.5, 0xb5652a);
    c.ellipse(10, 5, 3.5, 2.5, 0xe08a40);
    c.px(9, 4, 0xffd090);
    c.outline(OUT);
    return c;
}
function drawMagnet() {
    const c = new PixelCanvas(14, 14);
    c.ring(7, 6, 6, 3, 0xd03030);
    c.rect(1, 6, 3, 4, 0xd03030);
    c.rect(10, 6, 3, 4, 0xd03030);
    for (let y = 7; y < 14; y++) for (let x = 0; x < 14; x++) if (x > 3 && x < 10) c.clear(x, y);
    c.rect(1, 10, 3, 3, 0xd8dde8);
    c.rect(10, 10, 3, 3, 0xd8dde8);
    c.outline(OUT);
    return c;
}
function drawChest() {
    const c = new PixelCanvas(20, 16);
    c.rect(1, 6, 18, 9, 0x8a4a20);
    c.rect(1, 2, 18, 4, 0xa85a28);
    c.rect(1, 2, 18, 1, 0xc87a40);
    c.rect(1, 6, 18, 1, 0xf5c542);
    c.rect(9, 5, 2, 4, 0xf5c542);
    c.rect(1, 2, 1, 13, 0xf5c542);
    c.rect(18, 2, 1, 13, 0xf5c542);
    c.outline(OUT);
    return c;
}
// ---------------------------------------------------------------- 场景
function drawGround() {
    const N = 64;
    const c = new PixelCanvas(N, N);
    const r = rng(1234);
    const cols = [0x3e6b34, 0x3a6531, 0x426f37];
    for (let y = 0; y < N; y++) for (let x = 0; x < N; x++) c.px(x, y, cols[Math.floor(r() * 3)]);
    const wrap = (x: number, y: number, col: number) => c.px((x + N) % N, (y + N) % N, col);
```

```
    // 深色斑块
    for (let k = 0; k < 5; k++) {
        const cx = Math.floor(r() * N), cy = Math.floor(r() * N), rr = 3 + r() * 4;
        for (let y = -8; y <= 8; y++) for (let x = -8; x <= 8; x++) {
            if (x * x + y * y < rr * rr && r() > 0.25) wrap(cx + x, cy + y, 0x355d2d);
        }
    }
    // 草丛
    for (let k = 0; k < 22; k++) {
        const x = Math.floor(r() * N), y = Math.floor(r() * N);
        wrap(x, y, 0x5a8f45); wrap(x, y - 1, 0x6aa04f);
        wrap(x + 2, y, 0x5a8f45); wrap(x + 2, y - 1, 0x5a8f45); wrap(x + 2, y - 2, 0x7ab55a);
        wrap(x + 1, y + 1, 0x2f5228);
    }
    // 小石子
    for (let k = 0; k < 6; k++) {
        const x = Math.floor(r() * N), y = Math.floor(r() * N);
        wrap(x, y, 0x6e6a5a); wrap(x + 1, y, 0x8a8674);
    }
    return c;
}
// ---------------------------------------------------------------- 冰雪荒原 / 熔岩火山 地图
function tileNoise(seed: number, cols: number[], specks: [number, number][], cracks?: { col: number; glow: numbe
    r; n: number }) {
    const N = 64;
    const c = new PixelCanvas(N, N);
    const r = rng(seed);
    for (let y = 0; y < N; y++) for (let x = 0; x < N; x++) c.px(x, y, cols[Math.floor(r() * cols.length)]);
    const wrap = (x: number, y: number, col: number) => c.px((x + N) % N, (y + N) % N, col);
    for (const [col, n] of specks) {
        for (let k = 0; k < n; k++) {
            const cx = Math.floor(r() * N), cy = Math.floor(r() * N), rr = 2 + r() * 4;
            for (let y = -6; y <= 6; y++) for (let x = -6; x <= 6; x++) if (x * x + y * y < rr * rr && r() > 0.3
    ) wrap(cx + x, cy + y, col);
        }
    }
    if (cracks) {
        for (let k = 0; k < cracks.n; k++) {
            let x = Math.floor(r() * N), y = Math.floor(r() * N);
            for (let s = 0; s < 14; s++) {
                wrap(x, y, cracks.col);
                if (s % 3 === 1) wrap(x + 1, y, cracks.glow);
                x += r() < 0.5 ? 1 : -1;
                y += r() < 0.7 ? 1 : 0;
            }
        }
    }
    return c;
}
function drawGroundSnow() {
```

```
    const c = tileNoise(77, [0xc9d5e3, 0xd2dce8, 0xc2cede], [[0xb0c0d4, 6], [0xdde6f0, 5]]);
    const r = rng(78);
    for (let k = 0; k < 10; k++) {
        const x = Math.floor(r() * 64), y = Math.floor(r() * 64);
        c.px(x, y, 0xa8bcd4); c.px(x + 1, y, 0xb8c8dc);
    }
    return c;
}
function drawGroundLava() {
    return tileNoise(91, [0x2a1a1c, 0x30201e, 0x261618], [[0x3a2422, 5], [0x1c1012, 4]], { col: 0xd04018, glow:
    0xffa030, n: 5 });
}
// ---- 毒雾沼泽
function drawGroundSwamp() {
    const c = tileNoise(131, [0x2c2a1c, 0x322f20, 0x282618], [[0x3a3a24, 5], [0x16222a, 6], [0x1c2c30, 4]]);
    const r = rng(132);
    for (let k = 0; k < 6; k++) {
        const x = Math.floor(r() * 60), y = Math.floor(r() * 60);
        c.px(x, y, 0x6a8a3a); c.px(x + 1, y, 0x55702e); c.px(x, y + 1, 0x55702e);
    }
    return c;
}
function drawDeadTree() {
    const c = new PixelCanvas(30, 40);
    c.rect(13, 14, 4, 26, 0x3a3028);
    c.rect(14, 14, 1, 26, 0x4a4034);
    c.line(15, 22, 5, 12, 0x3a3028, 255, 2);
    c.line(5, 12, 3, 5, 0x3a3028);
    c.line(15, 18, 25, 8, 0x3a3028, 255, 2);
    c.line(25, 8, 28, 3, 0x3a3028);
    c.line(20, 13, 23, 15, 0x3a3028);
    c.rect(10, 38, 10, 2, 0x2a2018);
    c.px(6, 11, 0x5a7a3a); c.px(24, 9, 0x5a7a3a); c.px(14, 30, 0x5a7a3a);
    c.outline(OUT);
    return c;
}
function drawReeds() {
    const c = new PixelCanvas(18, 24);
    for (const [x, h, col] of [[3, 16, 0x5a7a34], [7, 22, 0x6a8a3a], [11, 18, 0x5a7a34], [14, 13, 0x4a6a2e]] as
    [number, number, number][]) {
        c.line(x, 23, x + (x % 2 ? 1 : -1), 23 - h, col);
        if (h > 17) c.rect(x - 1, 23 - h, 2, 5, 0x6a4a2a);
    }
    c.outline(OUT);
    return c;
}
function drawMushroom() {
    const c = new PixelCanvas(20, 16);
    c.rect(5, 9, 3, 6, 0xe8dcc0);
    c.ellipse(6.5, 8, 5.5, 3.5, 0x9a40c0);
```

```
    c.px(4, 7, 0xe0b0ff); c.px(8, 6, 0xe0b0ff);
    c.rect(13, 11, 2, 4, 0xe8dcc0);
    c.ellipse(14, 10, 3.5, 2.5, 0x7ad040);
    c.px(13, 9, 0xd0ffa0);
    c.outline(OUT);
    return c;
}
function drawBogPool() {
    const c = new PixelCanvas(36, 18);
    c.ellipse(18, 9, 17.5, 8.5, 0x1a2a14);
    c.ellipse(18, 9, 15, 6.5, 0x3a6a24);
    c.ellipse(17, 8, 10, 4, 0x6aa030);
    c.circle(12, 7, 1.5, 0xb0e060); c.circle(22, 9, 1, 0xb0e060);
    return c;
}
function drawBubble() {
    const c = new PixelCanvas(20, 20);
    c.circle(10, 10, 8, 0x80e040, 200);
    c.circle(10, 10, 6, 0xa8ff60, 160);
    c.circle(7, 7, 2, 0xf0ffd0);
    c.outline(0x2a4a14);
    return c;
}
// ---- 圣诞：雪夜小镇
function drawGroundXmas() {
    const c = tileNoise(141, [0xd8e2ee, 0xe0e8f2, 0xd0dae8], [[0xc0cede, 5], [0xeef4fa, 6]]);
    const r = rng(142);
    // 零星的彩色灯光反射
    for (let k = 0; k < 5; k++) {
        const x = Math.floor(r() * 62), y = Math.floor(r() * 62);
        c.px(x, y, [0xff6060, 0x60d060, 0xffd040, 0x60a0ff][k % 4]);
    }
    return c;
}
function drawXmasTree() {
    const c = new PixelCanvas(32, 44);
    c.rect(14, 37, 4, 6, 0x6a4024);
    for (const [y, w] of [[30, 28], [22, 22], [14, 16], [7, 10]]) {
        c.poly([[16 - w / 2, y + 8], [16 + w / 2, y + 8], [16, y - 3]], 0x2a7a3a);
        c.poly([[16 - w / 2, y + 8], [16, y + 8], [16, y - 3]], 0x1e5a2c);
        c.rect(16 - w / 2 + 2, y + 6, w - 4, 1, 0xf0f6ff);
    }
    const r = rng(7);
    for (let k = 0; k < 9; k++) c.px(9 + Math.floor(r() * 14), 10 + Math.floor(r() * 26), [0xff4040, 0xffd040, 0
    x60a0ff][k % 3]);
    c.poly([[16, 0], [18, 4], [22, 4], [19, 7], [20, 11], [16, 8], [12, 11], [13, 7], [10, 4], [14, 4]], 0xffd04
    0);
    c.outline(OUT);
    return c;
}
```

```
function drawGift() {
    const c = new PixelCanvas(18, 18);
    c.rect(2, 7, 14, 10, 0xd83030);
    c.rect(1, 5, 16, 3, 0xe84848);
    c.rect(8, 5, 2, 12, 0xffd040);
    c.rect(1, 6, 16, 1, 0xffd040);
    c.ellipse(6, 3, 3, 2, 0xffd040);
    c.ellipse(12, 3, 3, 2, 0xffd040);
    c.outline(OUT);
    return c;
}
function drawCandyCane() {
    const c = new PixelCanvas(12, 26);
    for (let y = 6; y < 25; y++) c.rect(2, y, 3, 1, (y >> 1) % 2 ? 0xd83030 : 0xf8f8f8);
    // 拐杖弯钩：上半圆弧 + 右侧短尾
    for (let k = 0; k <= 16; k++) {
        const t = Math.PI + (k / 16) * Math.PI;
        const x = Math.round(6 + Math.cos(t) * 3.2), y = Math.round(6 + Math.sin(t) * 3.2);
        c.rect(x - 1, y, 3, 2, k % 4 < 2 ? 0xd83030 : 0xf8f8f8);
    }
    c.rect(8, 6, 3, 3, 0xd83030);
    c.outline(OUT);
    return c;
}
// 雪怪王（圣诞限定 Boss）：白色长毛雪怪 + 圣诞帽
function drawYeti(f: number) {
    const c = new PixelCanvas(52, 52);
    const W = 0xe8f0fa, WD = 0xb8c8dc, SKIN = 0x8aa0c0;
    const arm = f ? 2 : 0;
    c.ellipse(26, 34, 18, 16, WD);
    c.ellipse(26, 33, 17, 15, W);
    // 手臂
    c.ellipse(8, 32 + arm, 6, 11, WD);
    c.ellipse(44, 32 - arm, 6, 11, WD);
    c.ellipse(8, 31 + arm, 5, 10, W);
    c.ellipse(44, 31 - arm, 5, 10, W);
    // 脸
    c.ellipse(26, 24, 10, 8, SKIN);
    c.rect(20, 21, 3, 3, OUT); c.rect(29, 21, 3, 3, OUT);
    c.px(21, 21, 0x80e0ff); c.px(30, 21, 0x80e0ff);
    c.rect(21, 27, 10, 3, 0x3a2a40);
    c.px(22, 27, 0xffffff); c.px(29, 27, 0xffffff);
    // 毛发纹理
    for (const [x, y] of [[14, 40], [20, 44], [32, 44], [38, 40], [26, 46], [12, 30], [40, 28]] as [number, numb
    er][]) c.px(x, y, WD);
    // 脚
    c.ellipse(18, 49, 6, 3, WD); c.ellipse(34, 49, 6, 3, WD);
    // 圣诞帽
    c.poly([[13, 17], [39, 17], [30, 4], [24, 2]], 0xd83030);
    c.poly([[24, 2], [30, 4], [36, 0], [34, -1]], 0xd83030);
```

```
    c.rect(12, 15, 28, 4, 0xf8f8f8);
    c.circle(37, 1, 3, 0xf8f8f8);
    c.outline(OUT);
    return c;
}
// 圣诞蛙：圣诞帽 + 白胡子
function santaFrog(src: PixelCanvas) {
    const c = new PixelCanvas(src.w, src.h);
    c.data.set(src.data);
    const { top, cx, hw } = headBox(src);
    c.poly([[cx - hw, top + 6], [cx + hw, top + 6], [cx + 4, top - 8], [cx - 2, top - 9]], 0xd83030);
    c.poly([[cx + 4, top - 8], [cx + hw + 3, top - 3], [cx + hw + 4, top - 1]], 0xd83030);
    c.rect(Math.round(cx - hw - 1), top + 4, Math.round(hw * 2 + 2), 4, 0xf8f8f8);
    c.circle(cx + hw + 4, top, 2.5, 0xf8f8f8);
    // 胡子
    const by = top + Math.round(hw * 0.75) + 9;
    c.ellipse(cx, by + 3, hw * 0.7, 5, 0xf8f8f8);
    c.ellipse(cx, by + 1, hw * 0.45, 2, 0xe8e8f0);
    return c;
}
function drawPine() {
    const c = new PixelCanvas(28, 40);
    c.rect(12, 32, 4, 7, 0x5a3a24);
    for (const [y, w] of [[26, 24], [18, 18], [11, 12], [5, 7]]) {
        c.poly([[14 - w / 2, y + 8], [14 + w / 2, y + 8], [14, y - 2]], 0x2a5a48);
        c.poly([[14 - w / 2, y + 8], [14, y + 8], [14, y - 2]], 0x1e4436);
        c.rect(14 - w / 2 + 2, y + 6, w - 4, 2, 0xf0f6ff);
    }
    c.rect(13, 1, 2, 3, 0xf0f6ff);
    c.outline(OUT);
    return c;
}
function drawIceRock() {
    const c = new PixelCanvas(18, 14);
    c.poly([[1, 13], [4, 4], [9, 1], [14, 5], [17, 13]], 0x8ab8e0);
    c.poly([[4, 4], [9, 1], [9, 13], [4, 13]], 0xb8dcf8);
    c.line(9, 2, 12, 9, 0xe8f6ff);
    c.outline(OUT);
    return c;
}
function drawSnowman() {
    const c = new PixelCanvas(18, 24);
    c.circle(9, 17, 6.5, 0xf4f8ff);
    c.circle(9, 8, 4.5, 0xf4f8ff);
    c.circle(7.5, 16, 4.5, 0xdfe8f4);
    c.px(7, 7, OUT); c.px(11, 7, OUT);
    c.rect(9, 9, 3, 1, 0xff8a30);
    c.rect(4, 11, 10, 2, 0xd03030);
    c.rect(5, 1, 8, 3, 0x2a2a34);
    c.rect(3, 4, 12, 1, 0x2a2a34);
```

```
    c.outline(OUT);
    return c;
}
function drawLavaRock() {
    const c = new PixelCanvas(18, 14);
    c.ellipse(9, 8, 8, 5.5, 0x3a2626);
    c.ellipse(8, 7, 5.5, 3.5, 0x4e3430);
    c.line(4, 9, 9, 6, 0xe05018);
    c.line(9, 6, 13, 10, 0xff9030);
    c.outline(OUT);
    return c;
}
function drawObsidian() {
    const c = new PixelCanvas(14, 20);
    c.poly([[2, 19], [4, 6], [7, 0], [10, 7], [12, 19]], 0x241830);
    c.poly([[7, 0], [10, 7], [12, 19], [7, 19]], 0x3a2a50);
    c.line(5, 7, 6, 16, 0x8a6ac0);
    c.outline(OUT);
    return c;
}
function drawLavaPool() {
    const c = new PixelCanvas(36, 18);
    c.ellipse(18, 9, 17.5, 8.5, 0x5a1a10);
    c.ellipse(18, 9, 15, 6.5, 0xd04018);
    c.ellipse(17, 8, 10, 4, 0xff8a28);
    c.ellipse(15, 7, 4, 1.5, 0xffd060);
    return c;
}
function drawSkullPile() {
    const c = new PixelCanvas(18, 12);
    for (const [x, y] of [[5, 7], [12, 7], [9, 3]]) {
        c.circle(x, y, 3.2, 0xe8e2d0);
        c.px(x - 1, y, OUT); c.px(x + 1, y, OUT);
    }
    c.outline(OUT);
    return c;
}
function drawIcicle() {
    const c = new PixelCanvas(10, 24);
    c.poly([[1, 0], [9, 0], [5, 23]], 0x9ad4f8);
    c.poly([[3, 0], [6, 0], [5, 18]], 0xe8f8ff);
    c.outline(0x2a4a6a);
    return c;
}
function drawMeteor() {
    const c = new PixelCanvas(18, 26);
    c.poly([[5, 0], [13, 0], [15, 14], [3, 14]], 0xff8a20, 140);
    c.poly([[7, 2], [11, 2], [12, 14], [6, 14]], 0xffd060, 200);
    c.circle(9, 18, 6.5, 0x4a2a24);
    c.circle(8, 17, 4, 0x6a3a2a);
```

```
    c.px(10, 20, 0xff9030);
    c.outline(OUT);
    return c;
}
// ---------------------------------------------------------------- 新春活动：年夜庙会 / 年兽 / 福蛙
function drawGroundFest() {
    // 红砖广场：砖缝 + 金色花纹
    const N = 64;
    const c = new PixelCanvas(N, N);
    const r = rng(207);
    for (let y = 0; y < N; y++) for (let x = 0; x < N; x++) c.px(x, y, [0x7a2a26, 0x82302a, 0x742622][Math.floor
    (r() * 3)]);
    for (let y = 0; y < N; y += 16) {
        c.rect(0, y, N, 1, 0x4a1614);
        const off = (y / 16) % 2 ? 16 : 0;
        for (let x = off; x < N; x += 32) c.rect(x, y, 1, 16, 0x4a1614);
    }
    for (const [x, y] of [[12, 6], [44, 22], [28, 40], [56, 54]]) {
        c.rect(x - 2, y, 5, 1, 0xc89a40);
        c.rect(x, y - 2, 1, 5, 0xc89a40);
    }
    return c;
}
function drawLantern() {
    const c = new PixelCanvas(16, 36);
    c.rect(7, 0, 2, 36, 0x5a3a24);
    c.rect(2, 6, 12, 2, 0x5a3a24);
    c.rect(3, 8, 1, 3, 0xc89a40);
    c.ellipse(4, 15, 4, 5, 0xd82a20);
    c.ellipse(3.5, 14, 2, 3, 0xff6a40);
    c.rect(2, 10, 5, 1, 0xf5c542);
    c.rect(2, 19, 5, 1, 0xf5c542);
    c.rect(4, 20, 1, 4, 0xf5c542);
    c.rect(12, 8, 1, 3, 0xc89a40);
    c.ellipse(12, 15, 4, 5, 0xd82a20);
    c.ellipse(11.5, 14, 2, 3, 0xff6a40);
    c.rect(10, 10, 5, 1, 0xf5c542);
    c.rect(10, 19, 5, 1, 0xf5c542);
    c.rect(12, 20, 1, 4, 0xf5c542);
    c.outline(OUT);
    return c;
}
function drawFirecracker() {
    const c = new PixelCanvas(12, 28);
    c.rect(5, 0, 2, 28, 0x8a6a40);
    for (let i = 0; i < 6; i++) {
        const y = 3 + i * 4, x = i % 2 ? 6 : 1;
        c.rect(x, y, 5, 3, 0xd82a20);
        c.rect(x, y, 5, 1, 0xff6a40);
        c.px(x + (i % 2 ? 4 : 0), y + 1, 0xf5c542);
```

```
    }
    c.outline(OUT);
    return c;
}
function drawRedGate() {
    const c = new PixelCanvas(44, 40);
    c.rect(6, 10, 4, 30, 0xb02420);
    c.rect(34, 10, 4, 30, 0xb02420);
    c.rect(6, 10, 1, 30, 0x7a1814);
    c.rect(34, 10, 1, 30, 0x7a1814);
    c.poly([[0, 8], [44, 8], [40, 2], [4, 2]], 0x2a3a48);
    c.rect(2, 8, 40, 4, 0xd83020);
    c.rect(16, 13, 12, 7, 0xf5c542);
    c.rect(18, 15, 8, 3, 0xd83020);
    c.outline(OUT);
    return c;
}
function drawPlum() {
    const c = new PixelCanvas(28, 34);
    c.line(14, 33, 14, 18, 0x4a2a1a, 255, 2);
    c.line(14, 22, 6, 12, 0x4a2a1a);
    c.line(14, 20, 22, 9, 0x4a2a1a);
    c.line(8, 14, 4, 6, 0x4a2a1a);
    c.line(20, 11, 24, 4, 0x4a2a1a);
    const r = rng(33);
    for (let i = 0; i < 18; i++) {
        const x = 3 + r() * 22, y = 2 + r() * 18;
        c.circle(x, y, 1.6, r() < 0.5 ? 0xff7a9a : 0xffb0c4);
        c.px(x, y, 0xfff0a0);
    }
    c.outline(OUT, 200);
    return c;
}
function drawNian(f: number) {
    const c = new PixelCanvas(48, 48);
    const R = 0xd8402a, RD = 0x9a2418, GOLD = 0xf5c542, GD = 0xc08a20;
    // 腿
    const la = f ? 1 : -1;
    c.rect(12 + la, 36, 6, 10, RD);
    c.rect(30 - la, 36, 6, 10, R);
    c.rect(11 + la, 44, 8, 3, GOLD);
    c.rect(29 - la, 44, 8, 3, GOLD);
    // 身体
    c.ellipse(24, 30, 15, 10, R);
    c.ellipse(22, 32, 9, 6, 0xf07a4a);
    // 鬃毛
    for (let k = 0; k < 9; k++) {
        const a = Math.PI * (0.9 + k * 0.15);
        c.circle(24 + Math.cos(a) * 13, 17 + Math.sin(a) * 11, 4.5, k % 2 ? GOLD : GD);
    }
```

```
    // 头
    c.ellipse(24, 18, 11, 10, R);
    c.ellipse(24, 22, 8, 5, 0xf07a4a);
    // 角
    c.poly([[15, 10], [10, 0], [18, 8]], 0xf4ecd8);
    c.poly([[33, 10], [38, 0], [30, 8]], 0xf4ecd8);
    // 眼睛
    c.circle(19, 15, 3, 0xffffff);
    c.circle(29, 15, 3, 0xffffff);
    c.circle(19.5, 15.5, 1.6, OUT);
    c.circle(29.5, 15.5, 1.6, OUT);
    c.rect(15, 11, 7, 1, RD);
    c.rect(27, 11, 7, 1, RD);
    // 嘴
    if (f) {
        c.ellipse(24, 24, 6, 3.5, 0x4a0a0a);
        c.rect(19, 21, 2, 2, 0xffffff); c.rect(27, 21, 2, 2, 0xffffff);
    } else {
        c.rect(18, 24, 12, 2, 0x4a0a0a);
        c.px(19, 23, 0xffffff); c.px(28, 23, 0xffffff);
    }
    // 尾巴
    c.circle(41, 24, 4, GOLD);
    c.outline(OUT);
    return c;
}
// 福蛙：在奶蛙的头顶加瓜皮帽、胸前挂金钱
function fuFrog(src: PixelCanvas) {
    const c = new PixelCanvas(src.w, src.h);
    c.data.set(src.data);
    let top = -1, minX = src.w, maxX = 0;
    for (let y = 0; y < src.h && top < 0; y++) for (let x = 0; x < src.w; x++) if (src.alpha(x, y) > 0) { top =
    y; break; }
    for (let y = top; y < top + 6; y++) for (let x = 0; x < src.w; x++) if (src.alpha(x, y) > 0) { minX = Math.m
    in(minX, x); maxX = Math.max(maxX, x); }
    const cx = (minX + maxX) / 2;
    const hw = Math.max(8, (maxX - minX) / 2 + 2);
    c.ellipse(cx, top + 4, hw, 6, 0x9a1c18);
    c.rect(Math.round(cx - hw), top + 7, Math.round(hw * 2), 2, 0xf5c542);
    c.circle(cx, top - 2, 2.5, 0xf5c542);
    c.rect(Math.round(cx) - 1, top - 1, 2, 3, 0xd82a20);
    // 金钱
    const ny = Math.round(src.h * 0.52);
    c.circle(cx, ny, 4.5, 0xf5c542);
    c.rect(Math.round(cx) - 1, ny - 1, 3, 3, 0xc08a20);
    c.line(cx - 8, ny - 7, cx - 3, ny - 3, 0xd82a20);
    c.line(cx + 8, ny - 7, cx + 3, ny - 3, 0xd82a20);
    return c;
}
// 头顶位置（福蛙 / 德鲁伊蛙 / 棱镜蛙的装扮定位）
```

```
function headBox(src: PixelCanvas) {
    let top = -1, minX = src.w, maxX = 0;
    for (let y = 0; y < src.h && top < 0; y++) for (let x = 0; x < src.w; x++) if (src.alpha(x, y) > 0) { top =
    y; break; }
    for (let y = top; y < top + 6; y++) for (let x = 0; x < src.w; x++) if (src.alpha(x, y) > 0) { minX = Math.m
    in(minX, x); maxX = Math.max(maxX, x); }
    return { top, cx: (minX + maxX) / 2, hw: Math.max(8, (maxX - minX) / 2 + 2) };
}
// 德鲁伊蛙：树叶花冠 + 藤蔓披肩 + 小蘑菇
function druidFrog(src: PixelCanvas) {
    const c = new PixelCanvas(src.w, src.h);
    c.data.set(src.data);
    // 身体染成青绿色（黄色系像素 → 绿色系，保留明暗）
    const d = c.data;
    for (let i = 0; i < d.length; i += 4) {
        const r = d[i], g = d[i + 1], b = d[i + 2];
        if (d[i + 3] && r > 150 && g > 120 && b < g * 0.8) { d[i] = Math.round(r * 0.5); d[i + 1] = Math.round(g
     * 0.92); d[i + 2] = Math.round(b * 0.7 + 20); }
    }
    const { top, cx, hw } = headBox(src);
    for (let k = -3; k <= 3; k++) {
        const x = cx + k * (hw / 3.2), y = top + 2 + Math.abs(k) * 0.8;
        c.ellipse(x, y, 3, 2, k % 2 ? 0x3a9a40 : 0x58c050);
        c.px(Math.round(x), Math.round(y) - 1, 0xa0e080);
    }
    c.circle(cx, top - 1, 2, 0xe04040);
    c.px(Math.round(cx) - 1, top - 2, 0xffffff);
    // 藤蔓披肩
    const sy = Math.round(src.h * 0.5);
    for (let x = Math.round(cx - hw - 2); x <= cx + hw + 2; x++) {
        const y = sy + Math.round(Math.sin(x * 0.7) * 1.5);
        if (src.alpha(x, y) > 0) c.px(x, y, 0x2f7a35);
        if (src.alpha(x, y + 1) > 0) c.px(x, y + 1, 0x4a9a40);
        if (x % 4 === 0 && src.alpha(x, y + 2) > 0) c.px(x, y + 2, 0x7ad060);
    }
    return c;
}
// 棱镜蛙：青色护目镜 + 头顶水晶
function prismFrog(src: PixelCanvas) {
    const c = new PixelCanvas(src.w, src.h);
    c.data.set(src.data);
    const { top, cx, hw } = headBox(src);
    // 水晶
    c.poly([[cx, top - 9], [cx + 4, top - 3], [cx, top + 2], [cx - 4, top - 3]], 0x60e8ff);
    c.poly([[cx, top - 9], [cx + 4, top - 3], [cx, top - 3]], 0xd0ffff);
    c.line(cx, top + 2, cx, top + 4, 0x8090a0);
    // 护目镜：找眼睛所在行（头部偏下）
    const ey = top + Math.round(hw * 0.75) + 5;
    c.rect(Math.round(cx - hw - 1), ey - 3, Math.round(hw * 2 + 2), 7, 0x303848);
    c.rect(Math.round(cx - hw + 1), ey - 2, Math.round(hw - 2), 5, 0x40d8ff);
```

```
    c.rect(Math.round(cx + 1), ey - 2, Math.round(hw - 2), 5, 0x40d8ff);
    c.rect(Math.round(cx - hw + 2), ey - 1, 2, 2, 0xe0ffff);
    c.rect(Math.round(cx + 2), ey - 1, 2, 2, 0xe0ffff);
    return c;
}
function drawFirework() {
    const c = new PixelCanvas(24, 24);
    for (let k = 0; k < 12; k++) {
        const a = (k / 12) * Math.PI * 2;
        c.line(12 + Math.cos(a) * 4, 12 + Math.sin(a) * 4, 12 + Math.cos(a) * 11, 12 + Math.sin(a) * 11, k % 2 ?
     0xffd040 : 0xff5a40);
    }
    c.circle(12, 12, 2.5, 0xffffff);
    return c;
}
function drawTomb() {
    const c = new PixelCanvas(16, 22);
    c.rect(2, 7, 12, 13, 0x8a8a98);
    c.ellipse(8, 7, 6, 5, 0x8a8a98);
    c.rect(2, 4, 2, 16, 0x6a6a78);
    c.rect(7, 6, 2, 8, 0x5a5a66);
    c.rect(5, 8, 6, 2, 0x5a5a66);
    c.rect(0, 19, 16, 3, 0x4a3a28);
    c.px(11, 15, 0x5a8f45); c.px(12, 16, 0x5a8f45);
    c.outline(OUT);
    return c;
}
function drawTree() {
    const c = new PixelCanvas(36, 44);
    c.rect(15, 26, 6, 16, 0x5a3a20);
    c.rect(15, 26, 2, 16, 0x3e2814);
    c.rect(12, 40, 12, 2, 0x5a3a20);
    c.circle(18, 16, 12, 0x24452a);
    c.circle(11, 22, 8, 0x24452a);
    c.circle(25, 22, 8, 0x24452a);
    c.circle(17, 14, 10, 0x2f5a32);
    c.circle(22, 19, 7, 0x2f5a32);
    c.circle(15, 11, 6, 0x3e7540);
    c.circle(13, 9, 3, 0x56925a);
    c.outline(OUT);
    return c;
}
function drawBush() {
    const c = new PixelCanvas(20, 14);
    c.circle(6, 8, 5, 0x2f5a32);
    c.circle(13, 7, 6, 0x2f5a32);
    c.circle(11, 6, 4, 0x3e7540);
    c.px(10, 4, 0x56925a);
    c.px(5, 6, 0xd04050);
    c.px(15, 9, 0xd04050);
```

```
    c.outline(OUT);
    return c;
}
function drawRock() {
    const c = new PixelCanvas(14, 10);
    c.ellipse(7, 6, 6, 4, 0x6e6a60);
    c.ellipse(6, 5, 4, 2.5, 0x8e8a7e);
    c.px(5, 4, 0xaaa698);
    c.outline(OUT);
    return c;
}
function drawFlowers() {
    const c = new PixelCanvas(12, 8);
    const pts = [[2, 3, 0xf0e060], [6, 1, 0xf0f0f0], [9, 4, 0xe070a0], [4, 6, 0xf0f0f0]];
    for (const p of pts) {
        c.px(p[0], p[1] + 1, 0x2f5228);
        c.px(p[0] - 1, p[1], p[2]); c.px(p[0] + 1, p[1], p[2]);
        c.px(p[0], p[1] - 1, p[2]); c.px(p[0], p[1], 0xf0a020);
    }
    return c;
}
function drawBones() {
    const c = new PixelCanvas(14, 8);
    c.line(2, 6, 11, 2, 0xe8e2d0, 255, 1);
    c.line(2, 2, 11, 6, 0xd0c8b0, 255, 1);
    c.rect(1, 1, 2, 2, 0xe8e2d0); c.rect(10, 5, 2, 2, 0xe8e2d0);
    c.rect(1, 5, 2, 2, 0xd0c8b0); c.rect(10, 1, 2, 2, 0xd0c8b0);
    c.outline(OUT, 160);
    return c;
}
// ---------------------------------------------------------------- 图标 16x16
function icon(fn: (c: PixelCanvas) => void, outline = true) {
    const c = new PixelCanvas(16, 16);
    fn(c);
    if (outline) c.outline(OUT);
    return c;
}
const ICONS: Record<string, () => PixelCanvas> = {
    icon_whip: () => icon(c => {
        c.ring(9, 7, 5, 2, 0x8a5a30);
        c.line(3, 14, 6, 10, 0x4a2a14, 255, 2);
        c.px(13, 3, 0xc8a070);
    }),
    icon_wand: () => icon(c => {
        c.line(3, 13, 10, 6, 0x8a5a30, 255, 2);
        c.circle(11.5, 4.5, 3, 0x3fa9ff);
        c.px(11, 3, 0xffffff);
    }),
    icon_knife: () => icon(c => {
        c.line(2, 13, 5, 10, 0x6b3a1f, 255, 2);
```

```
        c.line(4, 9, 7, 12, 0xf5c542);
        c.line(7, 9, 13, 3, 0xd8dde8, 255, 2);
        c.px(14, 2, 0xffffff);
    }),
    icon_axe: () => icon(c => {
        c.line(3, 14, 11, 5, 0x8a5a30, 255, 2);
        c.poly([[8, 1], [15, 3], [15, 10], [11, 8], [8, 5]], 0xb8c0cc);
        c.line(14, 3, 14, 9, 0xffffff);
    }),
    icon_bible: () => icon(c => {
        c.rect(2, 1, 12, 14, 0x5a3aa0);
        c.rect(2, 1, 2, 14, 0x3a2470);
        c.rect(12, 2, 2, 12, 0xf4ecd8);
        c.poly([[8, 4], [9, 7], [12, 7], [9.5, 9], [10.5, 12], [8, 10], [5.5, 12], [6.5, 9], [4, 7], [7, 7]], 0x
    f5c542);
    }),
    icon_garlic: () => icon(c => {
        c.ellipse(8, 10, 6, 5, 0xf4ecd8);
        c.ellipse(8, 10, 2, 5, 0xd8ccb0);
        c.poly([[6, 6], [8, 1], [10, 6]], 0xe8dcc0);
        c.px(8, 1, 0x8ab060);
    }),
    icon_lightning: () => icon(c => {
        c.ring(8, 8, 7, 2, 0xf5c542);
        c.poly([[9, 2], [5, 9], [8, 9], [6, 14], [11, 7], [8, 7]], 0xfff080);
    }),
    icon_might: () => icon(c => {
        c.poly([[3, 13], [6, 4], [12, 2], [13, 8], [8, 14]], 0x3a9a40);
        c.line(4, 12, 11, 4, 0x6ad070);
    }),
    icon_armor: () => icon(c => {
        c.poly([[2, 2], [14, 2], [14, 8], [8, 15], [2, 8]], 0x9aa2b0);
        c.poly([[8, 2], [14, 2], [14, 8], [8, 15]], 0x7a8290);
        c.rect(7, 4, 2, 8, 0xf5c542);
    }),
    icon_heart: () => icon(c => {
        c.circle(5, 6, 3.5, 0xd02838);
        c.circle(11, 6, 3.5, 0xd02838);
        c.poly([[1.5, 7], [14.5, 7], [8, 14.5]], 0xd02838);
        c.px(4, 4, 0xff8080);
    }),
    icon_regen: () => icon(c => {
        c.circle(8, 9, 6, 0xe03828);
        c.circle(6, 7, 2, 0xff7a60);
        c.poly([[5, 2], [8, 4], [11, 2], [9, 5], [7, 5]], 0x3a9a40);
    }),
    icon_cooldown: () => icon(c => {
        c.rect(2, 2, 12, 12, 0x2a4a9a);
        c.rect(2, 2, 2, 12, 0x1a2a6a);
        c.rect(12, 3, 2, 10, 0xf4ecd8);
```

```
        c.rect(6, 6, 4, 4, 0xa0d0ff);
    }),
    icon_area: () => icon(c => {
        c.rect(7, 6, 2, 9, 0xf5c542);
        c.rect(3, 8, 10, 2, 0xf5c542);
        c.rect(3, 4, 2, 4, 0xf4ecd8);
        c.rect(11, 4, 2, 4, 0xf4ecd8);
        c.rect(7, 2, 2, 4, 0xf4ecd8);
        c.px(3, 2, 0xff9030); c.px(11, 2, 0xff9030); c.px(7, 0, 0xff9030);
    }),
    icon_speed: () => icon(c => {
        c.rect(2, 5, 12, 6, 0x8a5a30);
        c.rect(2, 7, 12, 2, 0xf5c542);
        c.rect(2, 5, 1, 6, 0x5a3a20);
    }),
    icon_move: () => icon(c => {
        c.poly([[2, 4], [14, 7], [10, 10], [4, 13]], 0xf0f0f8);
        c.line(4, 6, 12, 8, 0xc0c8e0);
        c.line(4, 9, 10, 9, 0xc0c8e0);
    }),
    icon_magnet: () => icon(c => {
        c.circle(8, 8, 6, 0x6a3ab0);
        c.circle(8, 8, 3, 0xa070f0);
        c.px(6, 5, 0xffffff);
    }),
    icon_amount: () => icon(c => {
        c.ring(6, 8, 4.5, 2, 0xf5c542);
        c.ring(10, 8, 4.5, 2, 0xd0a030);
    }),
    icon_duration: () => icon(c => {
        c.rect(4, 1, 8, 2, 0x8a5a30);
        c.rect(4, 13, 8, 2, 0x8a5a30);
        c.poly([[5, 3], [11, 3], [8, 8]], 0xe8d8a0);
        c.poly([[8, 8], [11, 13], [5, 13]], 0xe8d8a0);
        c.rect(6, 10, 4, 3, 0xd0a040);
    }),
    icon_growth: () => icon(c => {
        c.poly([[8, 1], [13, 7], [8, 15], [3, 7]], 0x3fd080);
        c.poly([[8, 1], [8, 7], [3, 7]], 0xa0ffc0);
    }),
    icon_fire: () => icon(c => {
        c.line(2, 14, 9, 7, 0x8a5a30, 255, 2);
        c.circle(11, 5, 4, 0xff6020);
        c.circle(11, 5, 2.5, 0xffc040);
        c.px(10, 4, 0xffffff);
    }),
    icon_holy: () => icon(c => {
        c.rect(6, 1, 4, 3, 0x8a5a30);
        c.ellipse(8, 10, 5.5, 5, 0x4080ff);
        c.ellipse(8, 9, 3.5, 3, 0x90c8ff);
```

```
        c.px(8, 8, 0xffffff); c.px(7, 9, 0xffffff); c.px(9, 9, 0xffffff); c.px(8, 10, 0xffffff);
        c.px(10, 11, 0xffffff); c.px(6, 12, 0xffffff);
    }),
    icon_cross: () => icon(c => {
        c.poly([[8, 0], [10, 6], [16, 8], [10, 10], [8, 16], [6, 10], [0, 8], [6, 6]], 0xf5c542);
        c.poly([[8, 2], [9, 7], [8, 8], [7, 7]], 0xfff0a0);
        c.circle(8, 8, 1.5, 0xfff8d0);
    }),
    icon_frost: () => icon(c => {
        for (let i = 0; i < 4; i++) {
            const a = (i / 4) * Math.PI;
            c.line(8 - Math.cos(a) * 6, 8 - Math.sin(a) * 6, 8 + Math.cos(a) * 6, 8 + Math.sin(a) * 6, 0x80e0ff,
     255, 2);
        }
        c.circle(8, 8, 2, 0xffffff);
    }),
    icon_familiar: () => icon(c => {
        c.circle(8, 8, 5.5, 0x60ff90, 140);
        c.circle(8, 8, 3.5, 0xa0ffc0);
        c.poly([[2, 4], [6, 7], [2, 10]], 0xd0ffe0);
        c.poly([[14, 4], [10, 7], [14, 10]], 0xd0ffe0);
        c.px(7, 7, OUT); c.px(9, 7, OUT);
    }),
    icon_chain: () => icon(c => {
        c.poly([[10, 0], [4, 7], [8, 7], [3, 15], [13, 6], [9, 6], [12, 0]], 0x9ad0ff);
        c.line(10, 1, 6, 6, 0xffffff);
    }),
    icon_laser: () => icon(c => {
        c.poly([[2, 8], [6, 3], [10, 8], [6, 13]], 0xa0f0ff);
        c.poly([[6, 3], [10, 8], [6, 8]], 0xffffff);
        c.rect(10, 7, 6, 2, 0x60e8ff);
        c.px(15, 7, 0xffffff);
    }),
    icon_spikes: () => icon(c => {
        c.rect(1, 13, 14, 2, 0x6a4a30);
        c.poly([[2, 13], [4, 5], [6, 13]], 0xb8a890);
        c.poly([[6, 13], [8, 1], [10, 13]], 0xd8ccb8);
        c.poly([[10, 13], [12, 6], [14, 13]], 0xb8a890);
    }),
    icon_bomb: () => icon(c => {
        c.circle(7, 10, 5.5, 0x2a2a3a);
        c.circle(5.5, 8.5, 1.5, 0x6a6a7a);
        c.rect(9, 3, 2, 3, 0x8a6a40);
        c.px(11, 2, 0xffc040); c.px(12, 1, 0xff6020); c.px(12, 3, 0xff6020);
    }),
    icon_luck: () => icon(c => {
        c.circle(5, 5, 3, 0x40c050);
        c.circle(11, 5, 3, 0x40c050);
        c.circle(5, 11, 3, 0x40c050);
        c.circle(11, 11, 3, 0x40c050);
```

```
        c.circle(8, 8, 2, 0x80e080);
        c.line(8, 10, 10, 15, 0x2a8a3a);
    }),
    icon_critdmg: () => icon(c => {
        c.poly([[8, 0], [10, 6], [16, 8], [10, 10], [8, 16], [6, 10], [0, 8], [6, 6]], 0xff8030);
        c.circle(8, 8, 2.5, 0xffe080);
    }),
    icon_fang: () => icon(c => {
        c.line(1, 2, 8, 5, 0x8a5a30); c.line(8, 5, 15, 2, 0x8a5a30);
        c.poly([[3, 3], [7, 4], [5, 14]], 0xf4ecd8);
        c.poly([[9, 4], [13, 3], [11, 14]], 0xf4ecd8);
        c.px(5, 12, 0x6ad070); c.px(11, 12, 0x6ad070);
    }),
    icon_thorns: () => icon(c => {
        c.poly([[2, 2], [14, 2], [14, 8], [8, 15], [2, 8]], 0x6a7a50);
        for (const [x, y] of [[1, 1], [15, 1], [0, 8], [16, 8], [8, 16], [8, 0]]) c.line(8, 8, x, y, 0x3a4a28, 2
    55, 1);
        c.rect(6, 5, 4, 4, 0xa0b070);
    }),
    icon_heal: () => drawChicken(),
};
function drawCoin() {
    const c = new PixelCanvas(10, 10);
    c.circle(5, 5, 4.5, 0xd8a020);
    c.circle(5, 5, 3.5, 0xf5c542);
    c.rect(4, 3, 2, 4, 0xd8a020);
    c.px(3, 2, 0xfff0a0);
    c.outline(OUT);
    return c;
}
function drawCoinBag() {
    const c = new PixelCanvas(16, 16);
    c.ellipse(8, 10, 6.5, 5.5, 0x9a6a30);
    c.rect(6, 2, 4, 4, 0x9a6a30);
    c.rect(5, 5, 6, 1, 0xf5c542);
    c.ellipse(6, 9, 2, 2, 0xc08a48);
    c.circle(8, 11, 2.5, 0xf5c542);
    c.outline(OUT);
    return c;
}
const UI_ICONS: Record<string, () => PixelCanvas> = {
    icon_pause: () => icon(c => { c.rect(3, 2, 4, 12, 0xffffff); c.rect(9, 2, 4, 12, 0xffffff); }),
    icon_gold: () => icon(c => {
        c.circle(8, 8, 6.5, 0xd8a020);
        c.circle(8, 8, 5, 0xf5c542);
        c.rect(7, 4, 2, 8, 0xd8a020);
        c.px(5, 4, 0xfff0a0);
    }),
    icon_reroll: () => icon(c => {
        c.ring(8, 8, 6.5, 2, 0x40c0ff);
```

```
        for (let y = 6; y < 11; y++) for (let x = 0; x < 6; x++) c.clear(x, y);
        c.poly([[0, 6], [6, 6], [3, 10]], 0x40c0ff);
    }),
    icon_revive: () => icon(c => {
        c.poly([[12, 1], [14, 4], [9, 11], [5, 13], [6, 9]], 0xf5c542);
        c.poly([[12, 1], [9, 6], [6, 9], [7, 7]], 0xfff0a0);
        c.line(5, 13, 2, 15, 0xc89020);
    }),
    icon_lock: () => icon(c => {
        c.ring(8, 6, 4.5, 2, 0xb0b8c8);
        c.rect(3, 7, 10, 8, 0xd0a030);
        c.rect(7, 9, 2, 3, 0x4a3010);
    }),
    icon_music: () => icon(c => {
        c.rect(6, 2, 2, 10, 0xffffff);
        c.rect(6, 2, 7, 2, 0xffffff);
        c.rect(11, 2, 2, 8, 0xffffff);
        c.ellipse(5, 12, 2.5, 2, 0xffffff);
        c.ellipse(10, 10, 2.5, 2, 0xffffff);
    }),
    icon_sound: () => icon(c => {
        c.rect(2, 6, 3, 4, 0xffffff);
        c.poly([[5, 6], [9, 2], [9, 14], [5, 10]], 0xffffff);
        c.line(11, 5, 11, 10, 0xffffff);
        c.line(13, 3, 13, 12, 0xffffff);
    }),
    icon_vibrate: () => icon(c => {
        c.rect(5, 2, 6, 12, 0xffffff);
        c.rect(6, 3, 4, 9, 0x40404a);
        c.line(2, 5, 2, 10, 0xffffff);
        c.line(13, 5, 13, 10, 0xffffff);
    }),
};
// 进化图标：金色描边 + 闪光
function evoIcon(base: PixelCanvas) {
    const c = new PixelCanvas(16, 16);
    c.data.set(base.data);
    const o = [(OUT >> 16) & 255, (OUT >> 8) & 255, OUT & 255];
    for (let i = 0; i < c.data.length; i += 4) {
        if (c.data[i + 3] && c.data[i] === o[0] && c.data[i + 1] === o[1] && c.data[i + 2] === o[2]) {
            c.data[i] = 0xff; c.data[i + 1] = 0xc0; c.data[i + 2] = 0x30;
        } else if (c.data[i + 3]) {
            // 偏红暖色调
            c.data[i] = Math.min(255, c.data[i] * 1.15 + 30);
            c.data[i + 1] = c.data[i + 1] * 0.8;
            c.data[i + 2] = c.data[i + 2] * 0.8;
        }
    }
    c.px(1, 1, 0xffffff); c.px(14, 2, 0xffffff); c.px(2, 14, 0xfff0a0);
    return c;
```

```
}
// ---------------------------------------------------------------- 新武器弹道 / 道具
function drawFireball() {
    const c = new PixelCanvas(16, 16);
    c.circle(8, 8, 7.5, 0xff4010, 120);
    c.circle(8, 8, 5.5, 0xff6020);
    c.circle(8, 8, 3.5, 0xffc040);
    c.circle(7, 7, 1.5, 0xffffff);
    return c;
}
function drawFlame(f: number) {
    const c = new PixelCanvas(32, 20);
    const r = rng(31 + f);
    c.ellipse(16, 15, 15, 5, 0xa02010, 120);
    for (let i = 0; i < 9; i++) {
        const x = 3 + i * 3.3 + r() * 2, h = 5 + r() * 9;
        c.poly([[x - 3, 17], [x + (f ? 1 : -1), 17 - h], [x + 3, 17]], 0xff6020);
        c.poly([[x - 1.5, 17], [x, 17 - h * 0.6], [x + 1.5, 17]], 0xffd040);
    }
    return c;
}
function drawFlask() {
    const c = new PixelCanvas(10, 14);
    c.rect(4, 0, 2, 3, 0x8a5a30);
    c.rect(3, 3, 4, 2, 0xc0e0ff);
    c.ellipse(5, 9, 4.5, 4.5, 0x4080ff);
    c.ellipse(4, 8, 1.5, 1.5, 0xc0e0ff);
    c.outline(OUT);
    return c;
}
function drawPuddle() {
    const c = new PixelCanvas(64, 40);
    c.ellipse(32, 20, 31.5, 19.5, 0x3070ff, 110);
    c.ellipse(32, 20, 26, 15, 0x60a0ff, 120);
    const r = rng(77);
    for (let i = 0; i < 10; i++) c.rect(8 + r() * 46, 6 + r() * 26, 2, 2, 0xe0f0ff, 220);
    return c;
}
function drawCrossProj() {
    const c = new PixelCanvas(18, 18);
    c.poly([[9, 0], [11, 7], [18, 9], [11, 11], [9, 18], [7, 11], [0, 9], [7, 7]], 0xf5c542);
    c.poly([[9, 2], [10, 8], [9, 9], [8, 8]], 0xfff0a0);
    c.circle(9, 9, 1.8, 0xfff8d0);
    c.outline(OUT);
    return c;
}
function drawFamiliar(f: number) {
    const c = new PixelCanvas(16, 16);
    c.circle(8, 8, 7.5, 0x40ff80, 80);
    c.circle(8, 8, 4.5, 0x80ffb0);
```

```
    c.circle(8, 8, 2.5, 0xe0ffe8);
    const wy = f ? 4 : 6;
    c.poly([[1, wy], [5, 8], [1, wy + 6]], 0xd0ffe0, 200);
    c.poly([[15, wy], [11, 8], [15, wy + 6]], 0xd0ffe0, 200);
    c.px(7, 7, OUT);
    c.px(9, 7, OUT);
    return c;
}
function drawFShot() {
    const c = new PixelCanvas(8, 8);
    c.circle(4, 4, 3.5, 0x60ff90, 160);
    c.circle(4, 4, 2, 0xe0ffe8);
    return c;
}
function drawBomb(f: number) {
    const c = new PixelCanvas(14, 16);
    c.circle(7, 10, 5.5, f ? 0xc03030 : 0x2a2a3a);
    c.circle(5.5, 8.5, 1.5, f ? 0xff8080 : 0x6a6a7a);
    c.rect(8, 2, 2, 4, 0x8a6a40);
    c.px(10, 1, 0xffc040);
    c.px(11, 0, f ? 0xffffff : 0xff6020);
    c.outline(OUT);
    return c;
}
function drawBrazier(f: number) {
    const c = new PixelCanvas(18, 26);
    c.rect(7, 14, 4, 10, 0x6a5040);
    c.rect(4, 23, 10, 2, 0x4a3828);
    c.poly([[2, 10], [16, 10], [13, 15], [5, 15]], 0x8a7050);
    c.rect(2, 10, 14, 1, 0xb09070);
    const h = f ? 9 : 7;
    c.poly([[4, 10], [6, 10 - h + 2], [9, 10 - h - 1], [12, 10 - h + 3], [14, 10]], 0xff6020);
    c.poly([[6, 10], [9, 10 - h + 2], [12, 10]], 0xffd040);
    c.outline(OUT);
    return c;
}
function drawWatch() {
    const c = new PixelCanvas(14, 16);
    c.rect(6, 0, 2, 3, 0xf5c542);
    c.circle(7, 9, 6, 0xf5c542);
    c.circle(7, 9, 4.5, 0xf4ecd8);
    c.line(7, 9, 7, 6, OUT);
    c.line(7, 9, 9, 10, OUT);
    c.outline(OUT);
    return c;
}
function drawNuke() {
    const c = new PixelCanvas(14, 18);
    c.rect(5, 0, 4, 18, 0xffe080);
    c.rect(0, 4, 14, 4, 0xffe080);
```

```
    c.rect(6, 1, 2, 16, 0xffffff);
    c.rect(1, 5, 12, 2, 0xffffff);
    c.outline(0xa07010);
    return c;
}
function drawPotion() {
    const c = new PixelCanvas(12, 16);
    c.rect(4, 0, 4, 2, 0x8a5a30);
    c.rect(4, 2, 4, 3, 0xd0d0e0);
    c.ellipse(6, 10, 5.5, 5.5, 0xe02040);
    c.ellipse(4.5, 8.5, 1.5, 2, 0xff90a0);
    c.outline(OUT);
    return c;
}
function drawShieldPickup() {
    const c = new PixelCanvas(14, 16);
    c.poly([[1, 1], [13, 1], [13, 8], [7, 15], [1, 8]], 0x60a0ff);
    c.poly([[7, 1], [13, 1], [13, 8], [7, 15]], 0x4080e0);
    c.rect(6, 3, 2, 9, 0xffffff);
    c.rect(3, 6, 8, 2, 0xffffff);
    c.outline(OUT);
    return c;
}
// ---------------------------------------------------------------- 开屏 / 标题背景
function drawMoon() {
    const c = new PixelCanvas(80, 80);
    // 外层光晕（暖黄色满月）
    c.circle(40, 40, 39.5, 0xffe08a, 22);
    c.circle(40, 40, 35, 0xffe49a, 36);
    c.circle(40, 40, 30, 0xffe8aa, 56);
    // 月面
    c.circle(40, 40, 25, 0xe0b850);
    c.circle(38.5, 38.5, 23.5, 0xf6d76a);
    c.circle(36, 35, 16, 0xfbe38a);
    c.circle(33, 31, 7, 0xfff3c0);
    // 环形山
    for (const [x, y, r] of [[45, 47, 4.5], [31, 47, 3], [47, 32, 3], [38, 54, 2.2], [52, 41, 2]]) {
        c.circle(x, y, r, 0xd8ae48);
        c.circle(x - 0.8, y - 0.8, r * 0.6, 0xeac45c);
    }
    return c;
}
function drawStar() {
    const c = new PixelCanvas(5, 5);
    c.rect(2, 0, 1, 5, 0xbfd8ff, 170);
    c.rect(0, 2, 5, 1, 0xbfd8ff, 170);
    c.px(2, 2, 0xffffff);
    return c;
}
function drawEmber() {
```

```
    const c = new PixelCanvas(5, 5);
    c.circle(2.5, 2.5, 2.5, 0xff9a30, 110);
    c.px(2, 2, 0xffe080);
    return c;
}
function drawCloud() {
    const c = new PixelCanvas(56, 14);
    for (const [x, y, rx, ry] of [[12, 9, 11, 4], [26, 7, 13, 5], [40, 8, 12, 4.5], [48, 10, 8, 3]]) c.ellipse(x
    , y, rx, ry, 0x3a1428, 210);
    for (const [x, y, rx] of [[24, 4, 9], [38, 5, 8]]) c.ellipse(x, y, rx, 1.6, 0x8a2a40, 200);
    return c;
}
// 地平线剪影：起伏山丘 + 墓碑 + 枯树（横向无缝平铺，周期 128）
function drawHills() {
    const W = 128, H = 44;
    const c = new PixelCanvas(W, H);
    const ridge = (x: number) => 24 + Math.sin((x / W) * Math.PI * 2) * 5 + Math.sin((x / W) * Math.PI * 6 + 1)
    * 2.5;
    for (let x = 0; x < W; x++) {
        const top = Math.round(ridge(x));
        c.rect(x, top, 1, H - top, 0x1a0c22);
    }
    // 灌木丛与圆树剪影（温和的夜晚山丘）
    for (const gx of [18, 54, 97]) {
        const t = Math.round(ridge(gx));
        c.ellipse(gx + 2, t - 2, 5, 3.5, 0x1a0c22);
        c.ellipse(gx + 6, t - 1, 3.5, 2.5, 0x1a0c22);
    }
    for (const tx of [36, 76, 115]) {
        const t = Math.round(ridge(tx));
        c.rect(tx, t - 8, 2, 9, 0x1a0c22);
        c.circle(tx + 1, t - 13, tx === 76 ? 6 : 5, 0x1a0c22);
        c.circle(tx - 2, t - 10, 3.5, 0x1a0c22);
        c.circle(tx + 4, t - 10, 3.5, 0x1a0c22);
    }
    return c;
}
function white() {
    const c = new PixelCanvas(4, 4);
    c.rect(0, 0, 4, 4, 0xffffff);
    return c;
}
function drawShadow() {
    const c = new PixelCanvas(16, 6);
    c.ellipse(8, 3, 8, 3, 0x000000, 80);
    return c;
}
function drawJoyBase() {
    const c = new PixelCanvas(48, 48);
    c.circle(24, 24, 23.5, 0xffffff, 40);
```

```
    c.ring(24, 24, 23.5, 2, 0xffffff, 120);
    return c;
}
function drawJoyKnob() {
    const c = new PixelCanvas(20, 20);
    c.circle(10, 10, 9.5, 0xffffff, 150);
    c.ring(10, 10, 9.5, 2, 0xffffff, 220);
    return c;
}
// ---------------------------------------------------------------- 打包
export function allSprites(): Record<string, PixelCanvas> {
    const s: Record<string, PixelCanvas> = {};
    for (const k of HERO_KINDS) {
        const base = decodeHero(HERO_ART[k]);
        s['hero_' + k + '_0'] = heroFrame(base, 1, 1);
        s['hero_' + k + '_1'] = heroFrame(base, 1.06, 0.94);
        s['hero_' + k + '_2'] = heroFrame(base, 0.97, 1.04);
    }
    for (let f = 0; f < 3; f++) s['hero_fu_' + f] = fuFrog(s['hero_hunter_' + f]);
    for (let f = 0; f < 3; f++) s['hero_druid_' + f] = druidFrog(s['hero_hunter_' + f]);
    for (let f = 0; f < 3; f++) s['hero_prism_' + f] = prismFrog(s['hero_hunter_' + f]);
    for (let f = 0; f < 3; f++) s['hero_santa_' + f] = santaFrog(s['hero_hunter_' + f]);
    for (let f = 0; f < 2; f++) {
        s['bat_' + f] = drawBat(f);
        s['slime_' + f] = drawSlime(f);
        s['zombie_' + f] = drawZombie(f);
        s['skeleton_' + f] = drawSkeleton(f);
        s['ghost_' + f] = drawGhost(f);
        s['reaper_' + f] = drawReaper(f);
        s['rat_' + f] = drawRat(f);
        s['imp_' + f] = drawImp(f);
        s['mummy_' + f] = drawMummy(f);
        s['shroom_' + f] = drawShroom(f);
        s['slimeking_' + f] = drawSlimeKing(f);
        s['bonelord_' + f] = drawBoneLord(f);
        s['vampire_' + f] = drawVampire(f);
        s['lich_' + f] = drawLich(f);
        s['eye_' + f] = drawEye(f);
        s['toad_' + f] = drawToad(f);
        s['yeti_' + f] = drawYeti(f);
    }
    for (const k of ['bone', 'blood', 'ice', 'darkorb', 'slimeball', 'scythe']) s['shot_' + k] = drawEnemyShot(k
    );
    s.warn = drawWarn();
    for (const k of ['bat', 'slime', 'zombie', 'skeleton', 'ghost', 'reaper', 'rat', 'imp', 'mummy', 'shroom', '
    slimeking', 'bonelord', 'vampire', 'lich', 'eye', 'toad', 'yeti']) {
        for (let f = 0; f < 2; f++) s[k + '_' + f + '_w'] = silhouette(s[k + '_' + f]);
    }
    s.bolt = drawBolt();
    s.knife = drawKnife();
```

```
s.axe = drawAxe();
s.bible = drawBible();
s.lightning = drawLightning();
s.spark = drawSpark();
s.whip = drawWhip();
s.aura = drawAura();
s.gem_blue = drawGem(0x3f8cff, 0xa8d4ff, 0x1f4fb0);
s.gem_green = drawGem(0x3fd060, 0xa8f0b0, 0x1f8a30);
s.gem_red = drawGem(0xff4050, 0xffa8b0, 0xa01828);
s.chicken = drawChicken();
s.magnet = drawMagnet();
s.chest = drawChest();
s.ground = drawGround();
s.ground_snow = drawGroundSnow();
s.ground_lava = drawGroundLava();
s.pine = drawPine();
s.icerock = drawIceRock();
s.snowman = drawSnowman();
s.lavarock = drawLavaRock();
s.obsidian = drawObsidian();
s.lavapool = drawLavaPool();
s.skullpile = drawSkullPile();
s.icicle = drawIcicle();
s.ground_fest = drawGroundFest();
s.ground_swamp = drawGroundSwamp();
s.ground_xmas = drawGroundXmas();
s.xmastree = drawXmasTree();
s.gift = drawGift();
s.candycane = drawCandyCane();
s.deadtree = drawDeadTree();
s.reeds = drawReeds();
s.mushroom = drawMushroom();
s.bogpool = drawBogPool();
s.bubble = drawBubble();
s.lantern = drawLantern();
s.firecracker = drawFirecracker();
s.redgate = drawRedGate();
s.plum = drawPlum();
s.firework = drawFirework();
for (let f = 0; f < 2; f++) { s['nian_' + f] = drawNian(f); s['nian_' + f + '_w'] = silhouette(s['nian_' + f
]); }
s.meteor = drawMeteor();
s.tomb = drawTomb();
s.tree = drawTree();
s.bush = drawBush();
s.rock = drawRock();
s.flowers = drawFlowers();
s.bones = drawBones();
s.white = white();
s.shadow = drawShadow();
```

```
    s.joy_base = drawJoyBase();
    s.joy_knob = drawJoyKnob();
    for (const k in ICONS) s[k] = ICONS[k]();
    for (const k in UI_ICONS) s[k] = UI_ICONS[k]();
    for (const w of ['whip', 'wand', 'knife', 'axe', 'bible', 'garlic', 'lightning']) s['icon_' + w + '_evo'] =
    evoIcon(s['icon_' + w]);
    for (const w of ['fire', 'holy', 'cross', 'frost', 'familiar', 'chain', 'bomb', 'laser', 'spikes']) s['icon_
    ' + w + '_evo'] = evoIcon(s['icon_' + w]);
    s.fireball = drawFireball();
    s.flame_0 = drawFlame(0);
    s.flame_1 = drawFlame(1);
    s.flask = drawFlask();
    s.puddle = drawPuddle();
    s.crossp = drawCrossProj();
    s.spike = drawSpike();
    s.spike_warn = drawSpikeWarn();
    s.familiar_0 = drawFamiliar(0);
    s.familiar_1 = drawFamiliar(1);
    s.fshot = drawFShot();
    s.bomb_0 = drawBomb(0);
    s.bomb_1 = drawBomb(1);
    s.brazier_0 = drawBrazier(0);
    s.brazier_1 = drawBrazier(1);
    s.brazier_0_w = silhouette(s.brazier_0);
    s.brazier_1_w = silhouette(s.brazier_1);
    s.watch = drawWatch();
    s.nuke = drawNuke();
    s.potion = drawPotion();
    s.shield = drawShieldPickup();
    s.moon = drawMoon();
    s.star = drawStar();
    s.ember = drawEmber();
    s.cloud = drawCloud();
    s.hills = drawHills();
    s.coin = drawCoin();
    s.coinbag = drawCoinBag();
    return s;
}
export interface AtlasRect { x: number; y: number; w: number; h: number; }
export function buildAtlas(size = 1024) {
    const sprites = allSprites();
    const names = Object.keys(sprites).sort((a, b) => sprites[b].h - sprites[a].h);
    const data = new Uint8Array(size * size * 4);
    const rects: Record<string, AtlasRect> = {};
    const PAD = 2;
    let x = PAD, y = PAD, rowH = 0;
    for (const n of names) {
        const c = sprites[n];
        if (x + c.w + PAD > size) { x = PAD; y += rowH + PAD; rowH = 0; }
        if (y + c.h + PAD > size) throw new Error('atlas overflow');
```

```
        for (let j = 0; j < c.h; j++) {
            const src = j * c.w * 4;
            data.set(c.data.subarray(src, src + c.w * 4), ((y + j) * size + x) * 4);
        }
        rects[n] = { x, y, w: c.w, h: c.h };
        x += c.w + PAD;
        rowH = Math.max(rowH, c.h);
    }
    return { size, data, rects };
}
```